FOUNDATIONS OF THEORETICAL COMPUTER SCIENCE

理论计算机科学基础

课程讲义

形式语言与自动机
图灵机与可计算性
归约、自我引用与随机性

2026 年秋季

目录

第 1 章　正则语言 **1**

1.1　字母表、串与语言 1

1.2　确定性有限自动机 DFA 2

1.3　Myhill–Nerode 引理与最小 DFA 3

1.4　非确定性有限自动机 NFA 5

1.5　正则语言的封闭性 6

1.6　正则表达式 REX 8

1.7　正则语言的泵引理 9

第 2 章　上下文无关语言 **10**

2.1　上下文无关文法 CFG 10

2.2　歧义性 12

2.3　CFL 的基本构造 13

2.4　下推自动机 PDA 14

2.5　乔姆斯基范式 CNF 16

2.6　CFL 的泵引理 18

2.7　CFL 对交与补不封闭 19

第 3 章　图灵机 **20**

3.1　DTM 的定义与描述 20

3.2　图灵机的构造例子 21

3.3　转换器、判定器、识别器与枚举器 23

3.4　格局与计算历史 24

3.5　TM 的变体 25

3.6　丘奇–图灵论题 26

第 4 章 不可判定性 **27**

4.1 可识别、可判定与补可识别 27
4.2 枚举与识别 27
4.3 编码与通用图灵机 28
4.4 接受问题与对角化 29
4.5 可判定语言的封闭性 30
4.6 可识别与补可识别语言的封闭性 31

第 5 章 归约 **31**

5.1 映射归约 31
5.2 接受、停机与空性 32
5.3 满性、等价与包含 33
5.4 CFG 的成员资格与空性 34
5.5 计算历史与 CFG 满性 35
5.6 LBA 的成员资格、空性与满性 36
5.7 更多语言性质与 Rice 定理 37
5.8 Kleene 谱系 39
5.9 典型判定问题对照 42

第 6 章 可计算性高级专题 **43**

6.1 自我复制与递归定理 43
6.2 递归定理的不可判定性应用 45
6.3 谕示图灵机与图灵归约 46
6.4 谕示与 Kleene 谱系 47
6.5 Kolmogorov 复杂性 49
6.6 $K(x)$ 的不可计算性 50
6.7 随机串语言 RANDOM 50

第 1 章　正则语言

有限自动机用有限个状态记录已经读过的输入。我们先研究它能够记住什么，再用不可区分关系刻画所需的状态数，最后建立 DFA、NFA 和正则表达式之间的联系。

1.1　字母表、串与语言

字母表 Σ 是一个有限的符号集合，例如 $\{0,1\}$ 或 $\{a,b\}$。一个串是这些符号组成的有限序列。串 w 的长度记作 $|w|$；长度为零的串记作 ε，称为空串。Σ^n 表示长度为 n 的全部串，因而

$$\Sigma^0=\{\varepsilon\},\qquad \Sigma^*=\bigcup_{n\ge 0}\Sigma^n,\qquad \Sigma^+=\Sigma^*\setminus\{\varepsilon\}.$$

语言就是串的集合，即 $L\subseteq\Sigma^*$。全文用 $\mathbb{N}=\{0,1,2,\dots\}$。补语言总是相对于当前固定的字母表取补：$\overline{L}=\Sigma^*\setminus L$。

串的连接记作 uv：先写 u，再写 v。有 $u\varepsilon=\varepsilon u=u$，通常 uv 与 vu 不同。语言的连接和星运算定义为

$$AB=\{uv:u\in A,v\in B\},\qquad A^0=\{\varepsilon\},\quad A^{n+1}=A^nA,\quad A^*=\bigcup_{n\ge 0}A^n.$$

将串的字符顺序倒过来得到 w^R；语言的反转是 $L^R=\{w^R:w\in L\}$。这里上标 R 表示反转。

例 1.1　空串、空语言与连接

ε 是一个串，$\{\varepsilon\}$ 是只有一个元素的语言，$\varnothing$ 是没有元素的语言。取 $A=\{0,01\}$、$B=\{1,10\}$，逐对连接得到

$$AB=\{01,010,011,0110\}.$$

连接的单位元是语言 $\{\varepsilon\}$：$A\{\varepsilon\}=A$。由于空语言中没有可选的串，$A\varnothing=\varnothing$。星运算允许连接零个串，因此 $\varnothing^*=\{\varepsilon\}$。

例 1.2　子串、子序列与标准序

在 $w=0101$ 中，101 是连续的一段，因而是子串；001 可以由第 $1,3,4$ 个字符组成，因而是子序列，但不是子串。

取 $0<1$，按长度优先、同长时按字典序排列，得到

$$\varepsilon,0,1,00,01,10,11,000,001,\dots.$$

这种顺序称为标准序或短字典序。每个串之前只有有限多个串，所以按此顺序枚举能够走到任何指定的串。后面构造枚举器时会使用它。

1.2 确定性有限自动机 DFA

一个确定性有限自动机（DFA）是五元组

$$M = (Q, \Sigma, \delta, q_0, F).$$

其中 Q 是有限非空状态集，Σ 是输入字母表，$q_0 \in Q$ 是初始状态，$F \subseteq Q$ 是接受状态集，$\delta : Q \times \Sigma \to Q$ 是转移函数。每个状态读到每种输入符号后，都有唯一的下一状态。

状态图中，无来源的箭头指向初始状态，双圆表示接受状态，标有 a 的边 $p \to q$ 表示 $\delta(p, a) = q$。DFA 从左到右读完全部输入，读完时所在的状态决定是否接受。把 δ 扩展到串上：

$$\widehat{\delta}(q, \varepsilon) = q, \qquad \widehat{\delta}(q, wa) = \delta(\widehat{\delta}(q, w), a).$$

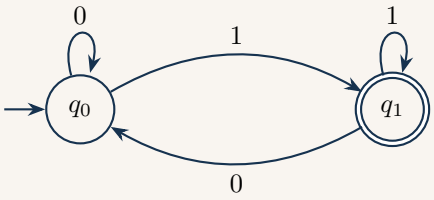
于是 M 接受的语言为

$$L(M) = \{w \in \Sigma^* : \widehat{\delta}(q_0, w) \in F\}.$$

能够被某个 DFA 接受的语言称为*正则语言*。

例 1.3　以 1 结尾的二进制串

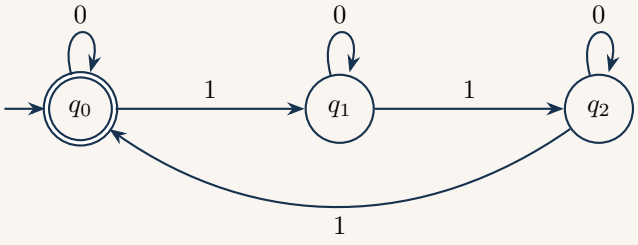
只需记住最后读到的字符是否为 1。令 $Q = \{q_0, q_1\}$，初始状态为 q_0，$F = \{q_1\}$。转移为 $\delta(q, 0) = q_0$、$\delta(q, 1) = q_1$，对两个状态 q 都成立。



输入 1011 的状态序列为 q_0, q_1, q_0, q_1, q_1，最终接受；输入 10 最终位于 q_0，拒绝。空串没有读入任何字符，停留在初始状态，也被拒绝。

例 1.4　记录 1 的个数模 3 的余数

令 q_i 表示已经读入的 1 的个数模 3 等于 i。读到 0 保持状态，读到 1 从 q_i 转到 $q_{(i+1) \bmod 3}$，以 q_0 为初态和唯一接受状态。



串 10101 含三个 1，所以接受。如果另加一个输入符号 `RESET`，并令所有状态读到它后转到 q_0，这台机器就能判断最后一次重置之后的 1 的个数是否为 3 的倍数。

例 1.5　首尾字符相同

在字母表 $\{a,b\}$ 上，考虑非空串的首尾字符是否相同。初始状态 s 表示尚未读入字符。读入字符后，使用四个状态 $q_{aa},q_{ab},q_{ba},q_{bb}$。下标的第一位记录首字符，第二位记录当前末字符。

$$\delta(s,a)=q_{aa},\quad \delta(s,b)=q_{bb},\qquad \delta(q_{uv},c)=q_{uc}.$$

接受状态为 q_{aa},q_{bb}。例如 $abba$ 的路径为 $s\to q_{aa}\to q_{ab}\to q_{ab}\to q_{aa}$。单字符串 a,b 都接受；初态 s 不接受，所以空串被排除。

1.3　Myhill–Nerode 引理与最小 DFA

不可区分关系

对于语言 L，在 Σ^* 上定义

$$x\sim_L y \iff \forall z\in\Sigma^*,\ [xz\in L \iff yz\in L]. \tag{1.1}$$

它表示读完 x 和读完 y 后，所有可能后缀给出的接受结果都相同。若存在某个 z 使两个结果不同，就称 z 为区分 x,y 的后缀。

这个关系具有自反性、对称性和传递性，因此是等价关系。它还满足*右不变性*：$x\sim_L y$ 蕴含 $xa\sim_L ya$。这是因为任取后缀 z，式 (1.1)可用于共同后缀 az。记 $[x]$ 为 x 所在的等价类，$\Sigma^*/\sim_L$ 为所有等价类组成的集合。

Myhill–Nerode 引理

语言 L 正则，当且仅当 $\sim_L$ 只有有限多个等价类。此时，识别 L 的最小 DFA 的状态数恰为这些等价类的个数。

证明。先设 DFA M 识别 L。如果 $\widehat{\delta}(q_0,x)=\widehat{\delta}(q_0,y)$，那么从这个相同状态继续读任何后缀 z，最终接受结果相同，故 $x\sim_L y$。所以不同的等价类必须到达不同状态，等价类个数至多为 $|Q|$。

反过来，若等价类个数有限，构造

$$Q_L=\Sigma^*/\sim_L,\quad q_L=[\varepsilon],\quad \delta_L([x],a)=[xa],\quad F_L=\{[x]:x\in L\}.$$

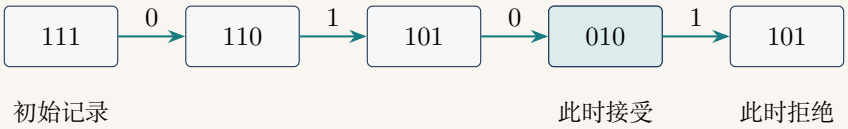
右不变性保证 δ_L 不依赖代表元的选择。取后缀 $z=\varepsilon$ 可知，同一类中的串要么都属于 L，要么都不属于，故 F_L 也定义良好。对输入长度归纳，读完 w 恰好到达 $[w]$，因此该 DFA 接受 L。

构造出的状态数等于等价类数，而第一部分说明任何 DFA 都至少需要这么多状态，所以它最小。每个类都由其代表串可达；任何最小 DFA 都与这个构造在状态重命名后相同。 □

例 1.6　倒数第 k 位为 0，需要多少状态

固定 $k\ge 1$，令 $L_k=\{w\in\{0,1\}^*:|w|\ge k,\ w\text{ 的倒数第}k\text{ 位为}0\}$。DFA 可以保存最后 k 位，初始记录设为 1^k；每读到 a，丢掉最左一位并在右侧添加 a。接受条件是记录的最左位为 0。当输入不足 k 位时，该位置仍是填充的 1，所以正确拒绝。这样得到 2^k 个状态。
为证最小，取任意不同的 $u,v\in\{0,1\}^k$，设它们从左数第 j 位不同。在两者后面都接 $z=1^{j-1}$，原来的第 j 位恰好成为倒数第 k 位。因此 uz,vz 中恰有一个属于 L_k。这 2^k 个串两两可区分，所以最小状态数为 2^k。

例 1.6　倒数第 k 位为 0，需要多少状态（续）



$k = 3$ 时逐个读入 0101。每一步保留最后三位，最左一位决定是否接受。

例 1.7　用无限多个等价类证明非正则

令 $L = \{0^n 1^n : n \geq 0\}$。考虑串 $\varepsilon, 0, 00, 000, \ldots$。若 $i \neq j$，后缀 1^i 将 0^i 与 0^j 区分开：$0^i 1^i \in L$，而 $0^j 1^i \notin L$。因此存在无限多个等价类，L 不是正则语言。

从 DFA 求最小 DFA

先删除从初始状态不可达的状态。对剩余状态定义 $p \equiv q$：从 p, q 出发读任意串，接受结果均相同。为了求这个关系，可以逐步细分状态分组。

1. 初始分组为 F 与 $Q \setminus F$，空组不保留。两组中的状态已经能被空串区分。
2. 对同组中的每个状态 q，列出所有 $\delta(q, a)$ 所在组的编号。这组编号称为状态的转移签名。
3. 将签名不同的状态分开，反复进行，直到分组不再改变。
4. 每个最终分组成为新状态，初态和接受状态按所属组确定，组间转移由原转移导出。

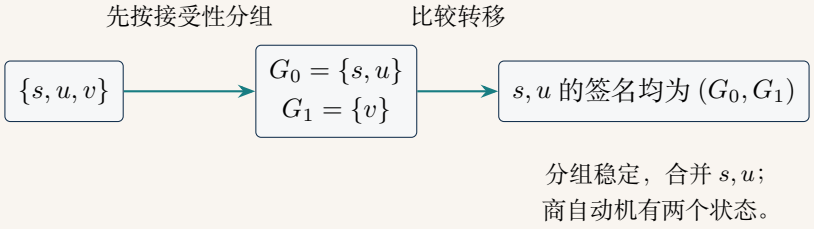
每次真正细分都会增加组数，而组数至多为 $|Q|$，因此算法终止。第 r 轮结束后，同组状态对长度不超过 r 的输入不能区分；若分组稳定，任意一步转移都保持同组，对输入长度归纳可知任何串都不能区分。于是最终分组恰好是 $\equiv$ 的等价类。

例 1.8　三状态 DFA 的合并

下面的 DFA 也识别以 1 结尾的串，初态是 s，唯一接受状态为 v。

状态	0	1
s	u	v
u	u	v
v	u	v

初始分组为 $\{s, u\}$、$\{v\}$。s, u 读 0 都进入第一组，读 1 都进入第二组，签名相同，分组已经稳定。合并 s, u 后得到前面的二状态 DFA。初始状态身份本身并不阻止两个状态合并。



1.4 非确定性有限自动机 NFA

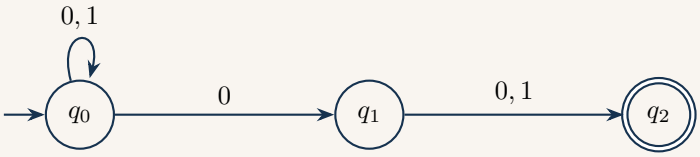
非确定性有限自动机（NFA）仍是五元组 $N=(Q,\Sigma,\delta,q_0,F)$，但转移函数变成

$$\delta : Q\times(\Sigma\cup\{\varepsilon\}) \longrightarrow \mathcal{P}(Q).$$

读到一个符号后，可以有零个、一个或多个后继状态；沿 ε 边移动时不消耗输入。输入 w 被接受的条件是：至少有一条从 q_0 出发、恰好读完 w 的路径到达接受状态。路径中的 ε 移动不计入读入的串。

例 1.9　猜测倒数第二位

要识别倒数第二位为 0 的串，可以一直留在 q_0，也可以在某次读到 0 时猜测它就是倒数第二位，转到 q_1，再读一个字符后到达 q_2。



对于 101，在第二个字符处离开 q_0 的路径能够接受。对于 1001，在第一个 0 处离开会过早到达 q_2，这条路径失败；在第二个 0 处离开则成功。一条路径失败并不影响另一条路径接受。一般的倒数第 k 位问题只需 $k+1$ 个 NFA 状态，而最小 DFA 需要 2^k 个状态。

ε 闭包与幂集构造

对 $S\subseteq Q$，用 $E(S)$ 表示从 S 中的状态出发，仅沿零条或多条 ε 边能够到达的状态集合，称为 ε 闭包。确定化时，一个 DFA 状态保存 NFA 当前可能所在的全部状态：

$$Q_D=\mathcal{P}(Q),\qquad s_D=E(\{q_0\}),\qquad F_D=\{S\subseteq Q : S\cap F\neq\varnothing\}.$$

对构造中实际出现的、已经取过闭包的集合 S，定义

$$\delta_D(S,a)=E\left(\bigcup_{q\in S}\delta(q,a)\right). \tag{1.2}$$

若希望在所有子集上统一采用“先做任意 ε 移动”的解释，也可把右侧的 S 换成 $E(S)$；从初态出发得到的可达部分相同。

证明。对已读入的串 w 的长度归纳。读入空串后，NFA 可以处于的状态恰为 $E(\{q_0\})$。若读完 w 的全部可能状态组成 S，那么再读 a 时，先从某个 $q\in S$ 沿一条 a 边移动，再做任意 ε 移动，得到的恰是式 (1.2)。所以 DFA 保存的集合始终准确；集合与 F 相交，当且仅当 NFA 存在接受路径。 □

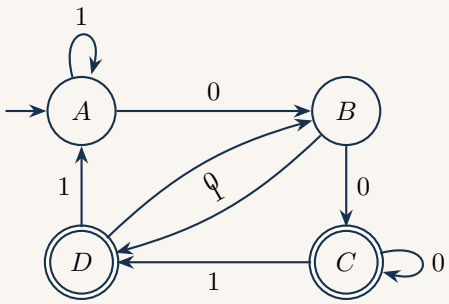
实际求解时，从初始集合开始用队列寻找可达集合即可，不必提前画出全部 $2^{|Q|}$ 个子集。空集合一旦出现，就是一个对所有字符都转向自身的拒绝状态。

例 1.10　把三状态 NFA 转成四状态 DFA

对上一例的 NFA，没有 ε 边，闭包就是集合本身。按式 (1.2)计算：

DFA 状态	读 0	读 1	接受?
$A=\{q_0\}$	B	A	否
$B=\{q_0,q_1\}$	C	D	否
$C=\{q_0,q_1,q_2\}$	C	D	是
$D=\{q_0,q_2\}$	B	A	是

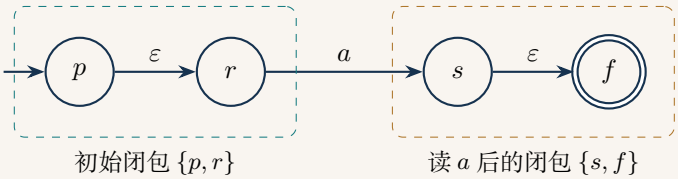
例如从 B 读 0，q_0 产生 q_0,q_1，q_1 产生 q_2，合并后为 C。这四个集合就是全部可达状态，和前面得到的最小状态数 2^2 一致。



每个圆中的字母代表表中的状态集合；含 q_2 的 C,D 画成双圆。

例 1.11　带 ε 边的状态集合

设初态 p 通过 ε 边到达 r，r 读 a 到 s，s 再通过 ε 边到达接受状态 f。则初始集合为 $\{p,r\}$；读入 a 后先得到 $\{s\}$，再取闭包得到 $\{s,f\}$，因此接受。ε 边要同时考虑在读字符之前和之后。



1.5　正则语言的封闭性

设 L_1,L_2 为正则语言。若两台机器的输入字母表不同，先统一到并集字母表，并为缺失的字符补上拒绝转移。

并、交与补

给定两个 DFA $M_i=(Q_i,\Sigma,\delta_i,s_i,F_i)$，用状态对同时记录它们的位置：

$$Q=Q_1\times Q_2,\quad s=(s_1,s_2),\quad \delta((p,q),a)=(\delta_1(p,a),\delta_2(q,a)).$$

识别交集时取 $F=F_1\times F_2$；识别并集时取 $F=(F_1\times Q_2)\cup(Q_1\times F_2)$。归纳可知读完任何 w，状态对的两个分量分别是原机器读完 w 的状态，所以接受条件恰好实现“且”与“或”。

对完整 DFA，将接受状态换成 $Q \setminus F$ 就得到补语言。完整性保证每个输入都能读到最后，原来接受和拒绝的结果因而交换。

例 1.12　偶数个 0 且以 1 结尾

第一台 DFA 记录 0 的个数奇偶，状态为 E, O；第二台记录是否以 1 结尾，状态为 Z, I。积自动机有四个状态，初态为 (E, Z)，唯一接受状态为 (E, I)。

$$(E,Z) \xrightarrow{0} (O,Z) \xrightarrow{0} (E,Z) \xrightarrow{1} (E,I).$$

因此 001 接受。若改为识别“偶数个 0 或以 1 结尾”，只需把接受状态改成 $(E,Z),(E,I),(O,I)$，转移保持相同。

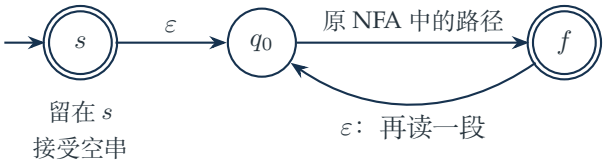
连接与星运算

用 NFA 可以直接连接计算路径。先将两台 NFA 的状态集改名为不相交的集合。对于 L_1L_2，从第一台的每个接受状态向第二台初态添加 ε 边，以第一台初态为总初态，只保留第二台的接受状态为总接受状态。接受路径在某个切分点从第一台转入第二台，恰好对应 $w = uv$、$u \in L_1, v \in L_2$。

对于 L^*，增加一个新的接受初态 s，从 s 向原初态添加 ε 边；从每个原接受状态向原初态添加 ε 边，保留原接受状态。停在 s 对应零次连接，每次返回原初态对应开始下一段。



连接构造：原 N_1 的接受状态作为接入点；
总机器的接受状态位于 N_2 中。



星运算的构造。图中 f 代表原机器的任一接受状态；回边连接下一段输入。

例 1.13　先读一段 0，再读一段 1

0^*1^* 可以由识别 0^* 和 1^* 的两个单状态 NFA 用 ε 边连接得到。输入 00011 在三个 0 后切换；输入 111 在开头切换；输入 ε 则通过 ε 边直接到达第二台的接受状态。输入 010 无法成功：一旦切到读 1 的部分，就没有读 0 的转移。

反转

把 NFA 的全部边反向，添加新的初态，通过 ε 边连到原来的每个接受状态，并把原初态作为唯一接受状态。原来读 $a_1 \cdots a_n$ 的接受路径反向后读 $a_n \cdots a_1$，所以得到 L^R。这也说明正则语言对反转封闭。

1.6 正则表达式 REX

正则表达式（REX）通过有限次递归构造描述语言。基本表达式是

$$\varnothing,\quad \varepsilon,\quad a\ (a\in\Sigma),$$

分别表示 $\varnothing,\{\varepsilon\},\{a\}$。若 R,S 是表达式，则 $R\cup S$、RS、R^* 也是表达式，含义分别是语言的并、连接与星运算。记表达式 R 描述的语言为 $L(R)$。通常星运算优先于连接，连接优先于并；括号可以明确运算顺序。

例 1.14　从语言要求写出表达式

在 $\Sigma=\{0,1\}$ 上：

表达式	表示的语言
$(0\cup1)^*01$	以 01 结尾的串
$1^*(01^*01^*)^*$	含偶数个 0 的串
$(0\cup1)^*0(0\cup1)(0\cup1)$	倒数第三位为 0 的串
$\varepsilon\cup0(0\cup1)^*$	空串，或首字符为 0 的非空串

第二个式子先允许任意多个 1，以后每个重复块恰好增加两个 0，中间及末尾可以夹任意多个 1。反过来，任意含偶数个 0 的串都可以把第 1, 2 个 0 配成一组、第 3, 4 个配成一组，从而按该式分块。

REX 转为 NFA

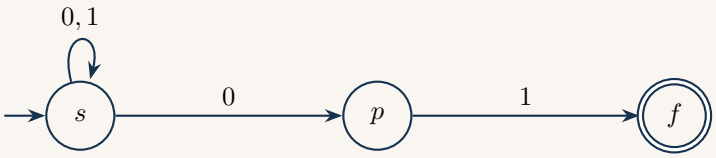
为每个表达式构造一个具有独立入口和出口的 NFA。基本表达式 a 对应一条 a 边，ε 对应一条 ε 边，$\varnothing$ 对应没有通路的入口和出口。复合表达式采用以下规则：

- $R\cup S$：新增入口，经 ε 边选择两个子机器之一；两个出口经 ε 边汇入新增出口。
- RS：把 R 的出口用 ε 边连到 S 的入口。
- R^*：新增入口和出口，允许直接通过；也允许进入 R，从其出口返回其入口，或转向新增出口。

对表达式的结构归纳，基本情况正确，每次组合又恰好实现对应语言运算，因此每个 REX 都描述一个正则语言。这种构造常称为 Thompson 构造。

例 1.15　为 $(0\cup1)^*01$ 构造 NFA

逐层使用上述规则能直接得到 NFA；合并冗余 ε 路径后，可以画成：



s 的循环读任意前缀，离开 s 的两条连续边读最后的 01。输入 1001 可以在第三个字符离开 s，最终接受。

NFA 转为 REX：状态消去

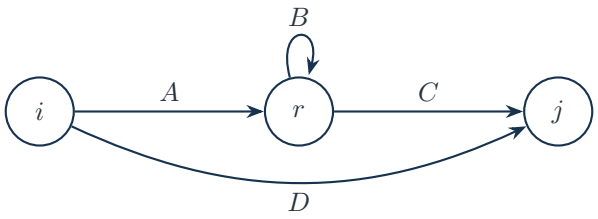
允许一条边标记整个正则表达式，就得到广义 NFA（GNFA）。先添加一个无入边的新初态 s、一个无出边的新终态 f；用 ε 边连接旧初态和旧终态。平行边用并合并，无边处视为标记 $\varnothing$。

消去中间状态 r 时，对所有保留下来的 i, j 更新

$$R'_{ij} = R_{ij} \cup R_{ir}(R_{rr})^* R_{rj}. \tag{1.3}$$

新边既保留原来的通路，也包含先进入 r、在 r 循环零次或多次、再离开到 j 的通路。逐个消去中间状态后，只剩 $s \to f$ 的一个表达式。

这一过程保持的不变量是：R_{ij} 描述从 i 到 j、内部只经过已经消去的状态的全部路径标签。式 (1.3)按是否经过新消去的 r 分解路径，所以不变量保持。最后它覆盖全部接受路径，证明每个 NFA 的语言都能由 REX 描述。



消去 r 后：$i \xrightarrow{D \cup AB^*C} j$

例 1.16　逐步消去一个两状态 DFA

识别以 1 结尾的 DFA 中，记拒绝状态为 q_0、接受状态为 q_1。添加 s, f 后，$s \to q_0$、$q_1 \to f$ 都标 ε。先消去 q_0：

$$R_{s,q_1} = 0^*1, \qquad R_{q_1,q_1} = 1 \cup 00^*1.$$

第二个式子包含原来的 1 自环，以及经 q_0 走一圈的 00^*1。再消去 q_1，得到

$$0^*1(1 \cup 00^*1)^*.$$

每个块都以 1 结尾；任何以 1 结尾的串又都能按相邻的 1 分成这样的块，因此它与较简洁的 $(0 \cup 1)^*1$ 等价。

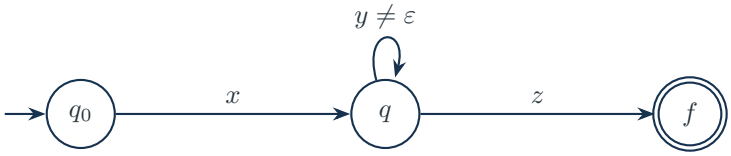
1.7　正则语言的泵引理

正则语言的泵引理

若 L 正则，则存在整数 $p \geq 1$，使每个满足 $w \in L$、$|w| \geq p$ 的串都可以写成 $w = xyz$，并且

$$|xy| \leq p, \qquad |y| > 0, \qquad \forall i \geq 0,\ xy^i z \in L.$$

证明。取识别 L 的 DFA，设状态数为 p。沿 w 的接受路径观察初态和读完前 $1, \ldots, p$ 个字符后的状态，总共有 $p+1$ 次状态出现。由抽屉原理，其中有两次相同，设位置为 $0 \leq r < s \leq p$。令 x 是前 r 个字符，y 是第 $r+1$ 到 s 个字符，剩余部分为 z。读 y 绕一圈回到同一状态，所以可以重复任意次，也可以完全跳过；随后读 z 仍到达原接受状态。其余两个长度条件直接由 $r < s \leq p$ 得到。 □



这里的 x, y, z 是路径片段的标签，可以包含多个字符。

使用泵引理证明非正则时，量词顺序决定证明方式：假设存在泵长度 p，选择一个依赖 p 的串 w；随后要对每一种满足长度条件的分解，找到一个破坏语言条件的泵次数 i。自己选出一个失败分解，还没有排除其他可能的成功分解。

例 1.17　0^n1^n 的泵引理证明

假设 $L = \{0^n1^n : n \geq 0\}$ 正则，取泵长度 p 和 $w = 0^p1^p$。任意合法分解 $w = xyz$ 都满足 $y = 0^k$，其中 $1 \leq k \leq p$。取 $i = 0$ 后

$$xz = 0^{p-k}1^p \notin L.$$

这与引理要求所有 $i \geq 0$ 都留在语言中矛盾。

例 1.18　长度是 2 的幂

考虑 $L = \{0^{2^n} : n \geq 0\}$。假设它正则，取 $2^m > p$，选择 $w = 0^{2^m}$。任意合法泵段长度 k 满足 $1 \leq k \leq p < 2^m$。泵一次额外的 y，得到长度 $2^m + k$，严格位于相邻的两个幂 2^m 与 2^{m+1} 之间，因而不属于 L。

例 1.19　长度是素数

设 $L = \{0^r : r \text{ 为素数}\}$，假设泵长度为 p，选素数 $r > p$。任意合法分解的泵段长度记为 $k \geq 1$。取 $i = r + 1$，泵后长度变成

$$r + (i-1)k = r + rk = r(1+k),$$

是合数。于是该语言非正则。

第 2 章　上下文无关语言

有限状态可以记录有限种信息。要描述任意深度的嵌套、匹配和递归结构，我们引入文法与栈：CFG 通过替换变量生成串，PDA 则用栈保存尚未完成的匹配。

2.1　上下文无关文法 CFG

上下文无关文法（CFG）是四元组

$$G = (V, \Sigma, R, S).$$

V 为有限的变量集，也称非终结符集；Σ 为有限终结符字母表，且 $V \cap \Sigma = \varnothing$；$S \in V$ 为起始变量；R 为有限产生式集合，每条产生式具有形式

$$A \to \alpha, \qquad A \in V, \quad \alpha \in (V \cup \Sigma)^*.$$

左侧只有一个变量，所以任何位置的 A 都可以用 α 替换，替换条件不依赖它周围的符号。

若 $\beta = uAv$、$\gamma = u\alpha v$，且 $A \to \alpha \in R$，记作 $\beta \Rightarrow \gamma$。零步或多步替换记作 $\Rightarrow^*$。文法生成的语言为

$$L(G) = \{w \in \Sigma^* : S \Rightarrow^* w\}.$$

由某个 CFG 生成的语言称为上下文无关语言（CFL）。

例 2.1　成对生成 0 和 1

文法 $S \to 0S1 \mid \varepsilon$ 生成 $\{0^n1^n : n \geq 0\}$。例如

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000S111 \Rightarrow 000111.$$

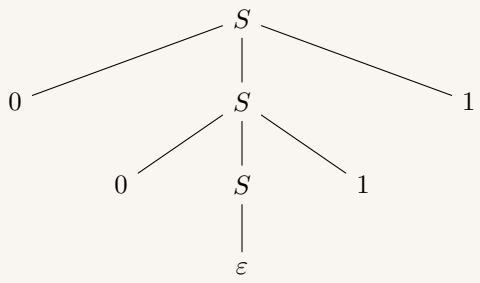
每次使用 $0S1$，都在左端增加一个 0、右端增加一个 1；最后用 ε 消去 S。因此所有生成串都满足目标形式。反过来，给定 n，恰好使用 n 次 $0S1$ 再使用一次 ε，就能生成 0^n1^n。

语法分析树与最左派生

语法分析树的根标为 S。若节点 A 使用产生式 $A \to X_1 \cdots X_k$，其子节点从左到右标为 $X_1, \ldots, X_k$；空产生式可以画一个 ε 叶子。从左到右读取全部终结符叶子，就得到生成串。

派生过程每一步都替换当前最左边的变量，称为最左派生。一棵固定的语法分析树可以对应不同的变量展开次序，但它唯一确定一个最左派生。因而判断歧义时，要比较不同分析树或不同最左派生。

例 2.2　0011 的语法分析树



叶子的非空符号依次是 $0, 0, 1, 1$。中间重复出现的 S 记录尚未配对的内部部分；每一层外侧的 0 与 1 是同一次产生式引入的一对。

例 2.3　平衡括号

文法

$$S \to (S)S \mid \varepsilon$$

例 2.3　平衡括号（续）

生成全部平衡括号串。非空平衡串可以唯一分解为 $(u)v$：第一个左括号与其匹配的右括号包住 u，其后剩余部分为 v，两者都平衡。这给出了文法正确性的递归证明，也说明了该文法的分解方式。例如

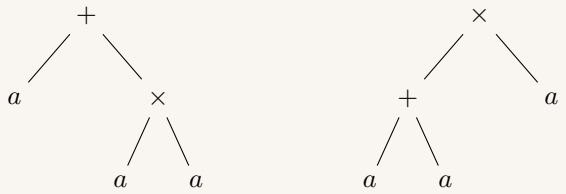
$$S \Rightarrow (S)S \Rightarrow ((S)S)S \Rightarrow (()S)S \Rightarrow (())S \Rightarrow (())(S)S \Rightarrow (())()S \Rightarrow (())().$$

2.2　歧义性

若某个串在 CFG G 中有两棵不同语法分析树，等价地有两个不同最左派生，则称 G 是*歧义文法*。这是对具体文法的性质描述。一个语言可能既有歧义文法，也有无歧义文法；若它的每个 CFG 都歧义，才称语言具有固有歧义。

例 2.4　加法与乘法的两种结构

文法 $E \to E + E \mid E \times E \mid a$ 对串 $a + a \times a$ 有两种分析结构：



$a + (a \times a)$　　$(a + a) \times a$

图中省略了部分变量节点，只保留运算结构。两个最左派生分别从 $E \Rightarrow E+E$ 与 $E \Rightarrow E \times E$ 开始，最终得到同一串。

例 2.5　用文法表达运算优先级

引入三类变量：

$$E \to E + T \mid T, \qquad T \to T \times F \mid F, \qquad F \to (E) \mid a.$$

E 管理加法，T 管理乘法，F 管理括号和原子。于是 $a + a \times a$ 的顶层只能按加法分开，右侧乘法保留在同一个 T 中；$a + a + a$ 则按左结合结构生成。改变文法后，优先级和结合方式都进入了语法结构。

2.3 CFL 的基本构造

并、连接、星与反转

给定 G_1, G_2，先将变量重命名，使 V_1, V_2 不相交，并取不属于它们的新起始变量 S。保留原产生式，分别增加

目标语言	新增产生式
$L_1 \cup L_2$	$S \to S_1 \mid S_2$
$L_1 L_2$	$S \to S_1 S_2$
L_1^*	$S \to S_1 S \mid \varepsilon$

新增产生式分别选择一个子文法、先后展开两个子文法，或连接有限多个 L_1 中的串。分析根节点的第一次展开，即可证明生成语言恰好等于目标语言。

对于反转，把每条 $A \to X_1 \cdots X_k$ 改成 $A \to X_k \cdots X_1$。这相当于将每棵分析树左右镜像，叶子的顺序随之反转，因此生成 L_1^R。

例 2.6　合并两个计数条件

令 $L_1 = \{0^n 1^n : n \ge 0\}$，$L_2 = \{1^m 2^m : m \ge 0\}$，分别用 $A \to 0A1 \mid \varepsilon$、$B \to 1B2 \mid \varepsilon$ 生成。增加 $S \to AB$ 后，得到

$$L = \{0^n 1^{n+m} 2^m : n, m \ge 0\}.$$

例如 $n = 2, m = 1$ 时，$S \Rightarrow AB \Rightarrow^* 0011B \Rightarrow^* 001112$。中间连续的 1 来自两个子文法，数量相加。

正则语言是 CFL

给定 DFA $M = (Q, \Sigma, \delta, q_0, F)$，为每个状态 q 引入变量 A_q。取 A_{q_0} 为起始变量；每条转移 $\delta(p, a) = q$ 对应产生式 $A_p \to aA_q$；每个接受状态再添加 $A_q \to \varepsilon$。

于是路径 $q_0 \xrightarrow{a_1} q_1 \cdots \xrightarrow{a_n} q_n \in F$ 恰好对应派生

$$A_{q_0} \Rightarrow a_1 A_{q_1} \Rightarrow \cdots \Rightarrow a_1 \cdots a_n A_{q_n} \Rightarrow a_1 \cdots a_n.$$

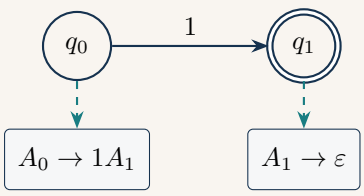
所以每个正则语言都有 CFG。这里得到的产生式至多在最右侧保留一个变量，称为右线性文法。

例 2.7　为以 1 结尾的语言写 CFG

使用第一章的二状态 DFA，得到

$$A_0 \to 0A_0 \mid 1A_1, \qquad A_1 \to 0A_0 \mid 1A_1 \mid \varepsilon,$$

起始变量为 A_0。例如 $A_0 \Rightarrow 1A_1 \Rightarrow 10A_0 \Rightarrow 101A_1 \Rightarrow 101$。只有到达 A_1 后才能停止派生，故终结串必以 1 结尾。



读字符的转移产生终结符；
接受状态允许结束派生。

2.4 下推自动机 PDA

下推自动机（PDA）在有限控制之外增加一个后进先出的栈。本节采用与单字符压栈、弹栈操作相适应的六元组：

$$P = (Q, \Sigma, \Gamma, \delta, q_0, F), \qquad \delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon),$$

其中 $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$、$\Gamma_\varepsilon = \Gamma \cup \{\varepsilon\}$，$\Gamma$ 为栈字母表，初始栈为空。转移

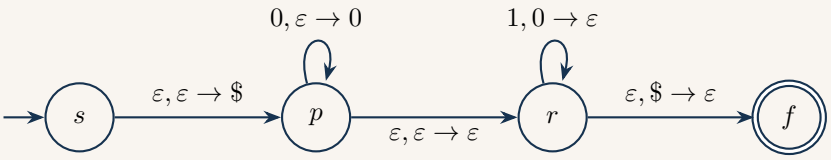
$$(q, b) \in \delta(p, a, c)$$

表示在状态 p 读 a，从栈顶弹出 c，压入 b，进入 q。某个位置写 ε 就表示省略相应读入、弹出或压入操作。特别地，$c = \varepsilon$ 表示不检查、不弹出栈顶，并不表示要求栈为空。

输入被接受，要求存在一条计算路径读完全部输入并到达 F。某些教材用初始栈符号和一次压入一个串的七元组定义；可以用初始步骤和若干中间状态互相转换。这里在图中用 $a, c \to b$ 标记边。

例 2.8　识别 0^n1^n 的 PDA

先压入栈底符号 \$。在状态 p 每读一个 0 就压入一个 0；通过 ε 边进入状态 r，随后每读一个 1 弹出一个 0。最后弹出 \$ 并进入接受状态。



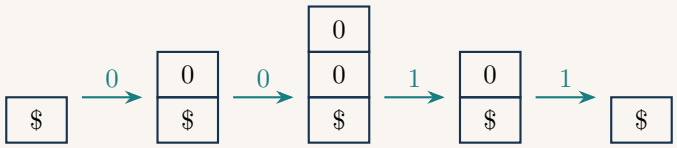
状态 r 没有读 0 的边，所以所有 0 都在 1 之前。栈保证两个字符数量相同。即使提前进入 f，剩余输入也无法读取，因而不构成接受计算。

例 2.9　0011 的栈变化

约定文字表示中栈顶在左。成功路径中，读入字符与栈的对应关系为

已读前缀	ε	0	00	001	0011
栈内容	\$	0\$	00\$	0\$	\$

最后用 ε 步弹出栈底符号。下图用竖直方向画同一个过程，最上端为栈顶。



输入 001 结束后仍有 0\$，不能弹出栈底；输入 011 在第二个 1 处找不到可弹出的 0。两者均拒绝。

例 2.10　回文中的非确定性

对 $\{ww^R : w \in \{0,1\}^*\}$，PDA 先把输入字符逐个压栈，再通过 ε 步猜测中点，随后逐个匹配并弹出栈顶。输入 0110 可在 01 后切换：栈顶先为 1，匹配下一个 1，再匹配最后的 0。
如果语言改成 $\{w\#w^R\}$，分隔符 $\#$ 明确给出切换位置；没有分隔符时，PDA 通过不同计算路径尝试中点。

CFG 转 PDA

PDA 的栈保存还没有与输入匹配的句型，栈顶放最左边的符号。先压入 \$ 和起始变量 S，随后在主状态重复两类操作：

1. 若栈顶为变量 A，非确定地选择产生式 $A \to X_1 \cdots X_k$，弹出 A，依次压入 $X_k, \ldots, X_1$，使 X_1 位于栈顶。
2. 若栈顶为终结符 a，读入一个 a 并弹出它。

看到栈底符号后可以弹出它并进入接受状态。长串压栈用一条宏边表示，实际可以拆成有限个单符号操作。

在任一成功路径中，已经匹配的输入前缀和栈中待处理部分共同组成当前句型。每次替换栈顶变量对应最左派生的一步，每次匹配终结符对应消耗其最左字符。由此，CFG 的终结派生与 PDA 的接受路径互相对应。

例 2.11　用栈模拟最左派生

对 $S \to 0S1 \mid \varepsilon$，匹配 0011 时的一条路径如下。栈顶仍写在左侧。

剩余输入	栈	操作
0011	$S\$$	展开 $S \to 0S1$
0011	$0S1\$$	读 0 并弹出 0
011	$S1\$$	再次展开 $S \to 0S1$
011	$0S11\$$	读 0 并弹出 0
11	$S11\$$	使用 $S \to \varepsilon$
11	$11\$$	连续匹配两个 1
ε	\$	弹出栈底并接受

这直接对应 $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 0011$。

PDA 转 CFG

先将 PDA 整理成只有一个接受状态、以空栈开始和结束、每步恰好压入或弹出一个符号的形式。无栈操作可以用压入再弹出一个新符号模拟，替换操作可以拆成弹出与压入；新增栈底符号和清栈阶段处理接受方式。

对状态对 p, q 引入变量 A_{pq}，表示使机器从状态 p 的某一栈高度出发，最终以相同栈内容到达 q、中间不动下层栈的输入串。添加：

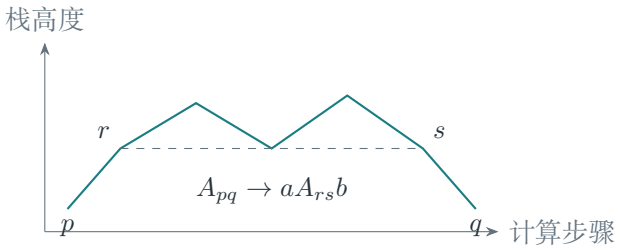
$$A_{pp} \to \varepsilon, \qquad A_{pq} \to A_{pr} A_{rq} \quad (r \in Q).$$

若从 p 到 r 的转移读 a 并压入 X，从 s 到 q 的转移读 b 并弹出 X，再添加

$$A_{pq} \to a A_{rs} b,$$

其中 a, b 可以为 ε。起始变量取 $A_{q_0 q_f}$。

一段栈高度回到原位的计算，要么可以在中途回到原位处拆成两段，要么最初压入的符号直到最后才弹出。前者对应连接规则，后者对应包裹规则。对计算长度归纳得到所有合法计算都能生成；反过来对派生树归纳，所有生成串都对应合法计算。这说明 PDA 与 CFG 描述同一语言类。



图中最初压入的符号支撑中间整段计算，直到最后一步才弹出。

CFL 与正则语言的交

设 PDA P 识别 CFL L，DFA D 识别正则语言 R。新 PDA 的状态同时保存两台机器的状态，栈沿用 P：P 读一个字符时，D 同时读该字符；P 做 ε 移动时，D 保持状态。读完输入后要求两个分量都接受，便得到 $L \cap R$。因此 CFL 对与正则语言取交封闭。

2.5 乔姆斯基范式 CNF

乔姆斯基范式（CNF）的产生式只有以下形式：

$$A \to BC, \qquad A \to a,$$

其中 A, B, C 为变量、a 为终结符；另外允许起始变量 $S_0 \to \varepsilon$，并要求 S_0 不出现在任何产生式右侧。这样既能表示空串，也让非空串的分析树具有清楚的二叉结构。

CNF 转换定理

每个 CFG 都能有效转换为一个生成相同语言的 CNF 文法，空串的归属也保持不变。

转换算法

1. **添加起始变量**。引入新的 S_0 和规则 $S_0 \to S$，保证起始变量不出现在右侧。
2. **求可空变量**。先标记所有具有 $A \to \varepsilon$ 的变量，再反复标记右侧全部由已标记变量组成的产生式左端，直到稳定。所得集合恰为能推出 ε 的变量。

3. **消去空产生式**。对每条规则，枚举其右侧可空变量的每一种删除组合，添加相应缩短规则；保留原规则。然后删除所有空产生式，仅在 S 可空时保留 $S_0 \to \varepsilon$。

4. **消去单一规则**。求每个 A 通过零条或多条 $X \to Y$ 可达的变量 B。把这些 B 的所有非单一规则复制给 A，然后一次性删除单一规则。

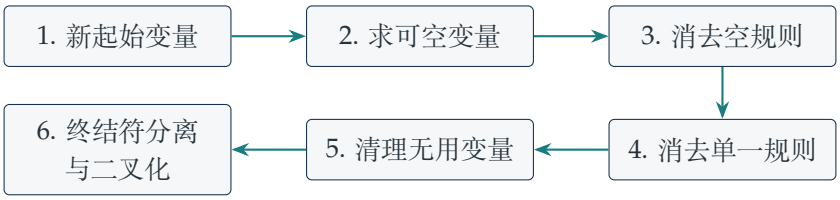
5. **清理无用变量**。先求能生成终结串的变量，删除其余变量及相关规则；再从 S_0 求可达变量，删除不可达部分。若语言为空，可用没有产生式的起始变量表示。

6. **分离终结符并拆长规则**。在长度至少为二的右侧，将终结符 a 换成新变量 T_a，并添加 $T_a \to a$。再把 $A \to X_1 \cdots X_k$（$k > 2$）拆成

$$A \to X_1 D_1, \quad D_1 \to X_2 D_2, \quad \ldots, \quad D_{k-2} \to X_{k-1} X_k.$$

每条长规则使用自己的新变量，或明确复用含义相同的部分。

可空变量的标记过程只在有限集合上增加元素，单一规则闭包也是有限图上的可达性计算，因而算法终止。删除可空变量相当于提前完成其空串派生；单一规则闭包相当于把连续变量替换一步完成；新引入的终结符变量和二叉链则只是细分原来的产生式。每一步的派生都可以互相展开或压缩，所以语言不变。



CNF 转换的先后顺序。消去空规则时产生的单一规则，在下一步统一处理。

例 2.12　包含空产生式和单一规则的转换

考虑

$$S \to AB \mid a, \qquad A \to aA \mid \varepsilon, \qquad B \to bB \mid b.$$

它生成 $\{a^i b^j : i \geq 0, j \geq 1\} \cup \{a\}$。可空变量只有 A。添加 S_0 并消去空产生式后：

$$S_0 \to S, \quad S \to AB \mid B \mid a, \quad A \to aA \mid a, \quad B \to bB \mid b.$$

从 S_0 可经单一规则到达 S, B，从 S 可到达 B。消去这些规则后，S_0, S 都获得右侧 AB, a, bB, b。此时 S 已不可达，可以删除。最终 CNF 为

$$\begin{aligned} &S_0 \to AB \mid a \mid T_b B \mid b, \\ &A \to T_a A \mid a, \qquad\qquad B \to T_b B \mid b, \\ &T_a \to a, \qquad\qquad T_b \to b. \end{aligned}$$

例如 $aabb$ 可由 $S_0 \Rightarrow AB \Rightarrow T_a AB \Rightarrow aAB \Rightarrow aaB \Rightarrow aaT_b B \Rightarrow aabB \Rightarrow aabb$ 生成。

例 2.13　拆开长规则并保留空串

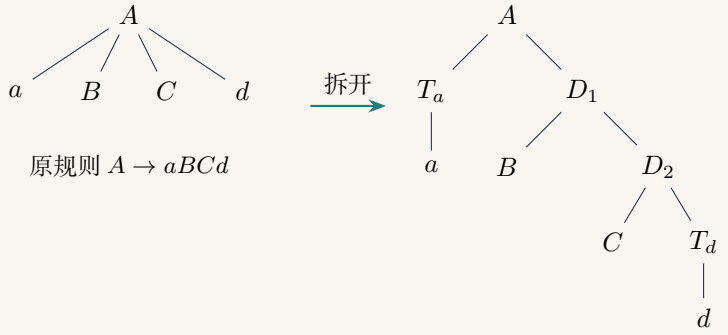
规则 $A \to aBCd$ 先改成 $A \to T_aBCT_d$，再拆为

$$A \to T_aD_1, \quad D_1 \to BD_2, \quad D_2 \to CT_d, \quad T_a \to a, \quad T_d \to d.$$

另一个常见例子是 $S \to aSb \mid \varepsilon$。一组等价 CNF 为

$$\begin{aligned} S_0 &\to T_aC \mid T_aT_b \mid \varepsilon, \\ S &\to T_aC \mid T_aT_b, \qquad C \to ST_b, \\ T_a &\to a, \qquad T_b \to b. \end{aligned}$$

$S_0 \to \varepsilon$ 保留空串；S 只生成正长度的 a^nb^n。S_0 从不出现在右侧。



原规则 $A \to aBCd$

两棵局部分析树的叶子顺序都是 a, B, C, d，新增变量只改变树的分支方式。

CNF 分析树的大小

对长度为 $n \geq 1$ 的终结串，CNF 分析树有 n 次 $A \to a$。去掉最后一层终结符后，变量节点形成满二叉树，有 n 个叶变量，因此有 $n-1$ 个二叉分支节点。总产生式使用次数恰为

$$(n-1) + n = 2n - 1.$$

这使固定输入的派生搜索成为有限过程，后面也会据此构造 CFL 成员资格判定算法。

2.6　CFL 的泵引理

上下文无关语言的泵引理

若 L 为 CFL，则存在 $p \geq 1$，使每个 $w \in L$、$|w| \geq p$ 都有分解 $w = uvxyz$，满足

$$|vxy| \leq p, \qquad |vy| > 0, \qquad \forall i \geq 0,\ uv^ixy^iz \in L.$$

这里 v 与 y 同时重复相同次数，其中一个允许为空，但不能同时为空。

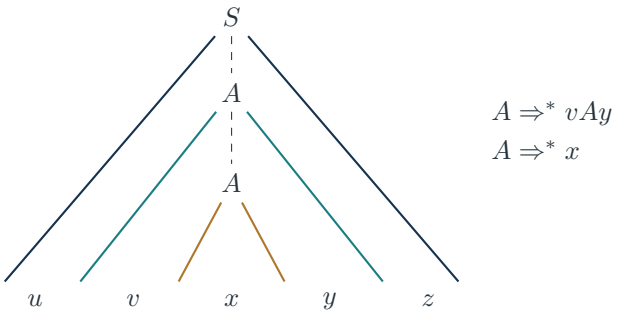
证明。把文法转换成 CNF，设变量个数为 m，取 $p = 2^{m+1}$。长度至少为 p 的串，其分析树必有一条根到叶路径含至少 $m+1$ 个变量节点；否则二叉树的叶子数不足 p。

取一条最长路径，在它靠近叶子的最后 $m+1$ 个变量节点中，有两个具有相同标记 A。上层 A 的子树对应串 vxy，下层 A 的子树对应串 x；整棵树其余左右部分分别为 u, z。于是

$$S \Rightarrow^* uAz, \qquad A \Rightarrow^* vAy, \qquad A \Rightarrow^* x.$$

上层 A 位于最长路径的最后 $m+1$ 个变量之内，其子树的叶子数至多为 $2^m \le p$，所以 $|vxy| \le p$。两次 A 之间至少有一次二叉分支，旁支生成非空串，故 $|vy| > 0$。

把中间的 $A \Rightarrow^* vAy$ 重复 i 次，再用下层派生 $A \Rightarrow^* x$ 收尾，就得到 uv^ixy^iz。取 $i = 0$ 对应直接用下层子树替换上层子树。 □



重复的变量让左右两侧的 v, y 成对增加，这正是 CFL 泵引理出现两个泵段的原因。

例 2.14　$0^n1^n2^n$ 不是 CFL

假设 $L = \{0^n1^n2^n : n \ge 0\}$ 为 CFL，取 $w = 0^p1^p2^p$。任意合法分解中，长度至多 p 的 vxy 不可能同时碰到 0 段和 2 段，否则必须包含中间全部 p 个 1 以及两端至少各一个字符。
所以至少一种字符在 v, y 中完全没有出现。取 $i = 0$，至少一个字符被删去，而未出现的那种字符仍有 p 个。三种字符数量不再相等，泵后的串不属于 L，矛盾。

例 2.15　复制语言 $\{ww\}$ 不是 CFL

令 $D = \{ww : w \in \{0,1\}^*\}$。若 D 是 CFL，则

$$D \cap 0^*1^*0^*1^* = \{0^i1^j0^i1^j : i, j \ge 0\}$$

也是 CFL。取测试串 $0^p1^p0^p1^p$。任意长度至多 p 的窗口 vxy 只能落在一个块或相邻两个块中。取 $i = 2$ 泵一次。如果某个泵段跨越 0/1 的边界，重复它会产生额外的字符块，破坏 $0^*1^*0^*1^*$ 的顺序；否则泵段都是单一字符，增加的字符只发生在至多两个相邻块中。第一、第三块或第二、第四块的相等关系至少有一组被破坏，因为对应的另一个块没有改变。于是所有合法分解都失败，得到矛盾。

2.7　CFL 对交与补不封闭

例 2.16　两个 CFL 的交不是 CFL

取

$$L_1 = \{0^n1^n2^m : n, m \ge 0\}, \qquad L_2 = \{0^m1^n2^n : m, n \ge 0\}.$$

L_1 是 $\{0^n1^n\}$ 与 2^* 的连接，L_2 是 0^* 与 $\{1^n2^n\}$ 的连接，所以两者都是 CFL。但

$$L_1 \cap L_2 = \{0^n1^n2^n : n \ge 0\}$$

不是 CFL。这个例子同时要求两对数量相等，超出了这两个单独文法各自处理的约束。

如果 CFL 对补封闭，由于它对并封闭，德摩根公式 $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$ 就会推出对交也封闭，与上一例矛盾。

例 2.17　一个 CFL 的补恰好是复制语言

令 $D=\{ww : w\in\{0,1\}^*\}$。用变量 A_0, A_1 分别生成中间字符为 $0,1$ 的所有奇数长度串：

$$A_b \to 0A_b0 \mid 0A_b1 \mid 1A_b0 \mid 1A_b1 \mid b \quad (b=0,1).$$

再令

$$S \to A_0 \mid A_1 \mid A_0A_1 \mid A_1A_0.$$

前两条生成全部奇数长度串。若串长为 $2n$ 且前后两半不同，选一对位置 i 与 $n+i$ 上不同的字符，在第 $2i-1$ 个字符后切开。两段长度分别为 $2i-1$ 与 $2(n-i)+1$，均为奇数；它们的中间字符正好是这对不同字符，因此属于 A_0A_1 或 A_1A_0。
反过来，两个奇数长度段的中心相距总长度的一半；若中心字符不同，前后两半就不可能相等。所以这个文法恰好生成 $\overline{D}$。它是 CFL，而其补 D 不是 CFL。

第 3 章　图灵机

PDA 只能直接访问栈顶。图灵机允许读写头在纸带上来回移动，并改写已经访问过的内容。我们先用具体机器理解读写操作，再讨论更方便的描述方式、机器变体和计算历史。

3.1　DTM 的定义与描述

确定性图灵机（DTM，通常简称 TM）是七元组

$$M=(Q,\Sigma,\Gamma,\delta,q_0,q_{\mathrm{acc}},q_{\mathrm{rej}}).$$

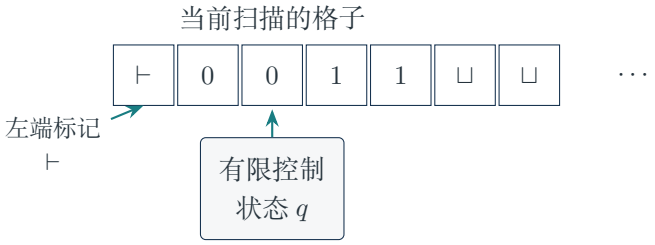
Q 为有限状态集，Σ 为输入字母表，Γ 为带字母表。空白符 $\sqcup\in\Gamma\setminus\Sigma$；初态为 q_0，两个不同的停机状态为 $q_{\mathrm{acc}}, q_{\mathrm{rej}}$。转移函数为

$$\delta : (Q\setminus\{q_{\mathrm{acc}},q_{\mathrm{rej}}\})\times\Gamma \to Q\times\Gamma\times\{L,R,S\}.$$

$\delta(q,a)=(r,b,L)$ 表示：当前读到 a 时，写入 b，进入状态 r，读写头左移一格；R,S 分别表示右移与不动。

采用向右无限延伸的纸带。为了便于展示算法，可以在第 0 格放左端标记 $\vdash$，输入从第 1 格开始，读写头初始指向第 1 格；机器保留左端标记并且不越过它。未用的格子为空白。左端标记是一种编码约定，可以用有限控制模拟，因此不改变模型的计算能力。

这里的 $\vdash$ 是放在纸带上的一个特殊符号，读作 “左端标记”，表示纸带的边界。它属于带字母表，既不是输入中的 0 或 1，也不是空白符 $\sqcup$。转移表中读到 $\vdash$ 的那一行，规定机器到达左端时怎样行动。



一次转移只读写一个格子，但可以重复扫描，从而比较距离很远的内容。

三种描述层次

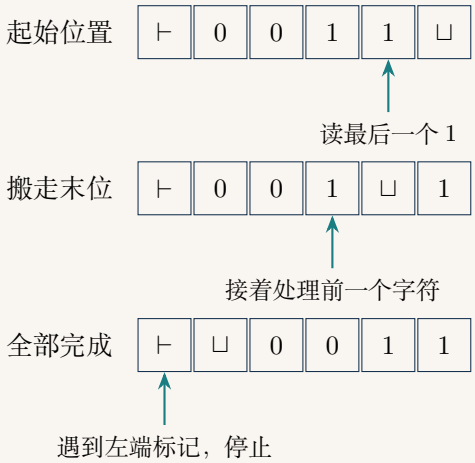
形式描述给出完整七元组、转移表或状态图；实现描述说明数据在带上的位置、标记方法和扫描次序；高层描述直接写有限的算法步骤，例如“模拟 M 在 w 上运行”。高层步骤应当能由 TM 实现。证明机器是判定器时，还要说明这些步骤在每个输入上都会终止。

例 3.1　将二进制串整体右移一格

设带上为 $\vdash 0011\sqcup\cdots$。先向右扫描到第一个空白，再从右向左处理每个原字符：读到字符 a 后，把当前格擦为空白，将 a 记在有限控制中；右移一格写入 a；再左移两格，处理前一个原字符。读到 $\vdash$ 时结束。关键带内容依次为

$$\vdash 0011\sqcup \longrightarrow \vdash 001\sqcup 1 \longrightarrow \vdash 00\sqcup 11 \longrightarrow \vdash 0\sqcup 011 \longrightarrow \vdash \sqcup 0011.$$

从右往左处理保证写入新位置时，该处原来的数据已经被搬走。空输入直接结束。要记忆的只是一个字符，因此可以用有限个状态完成。



每次把字符搬到右侧后，回到尚未搬动的前一格。左端标记一直保留在第 0 格。

3.2　图灵机的构造例子

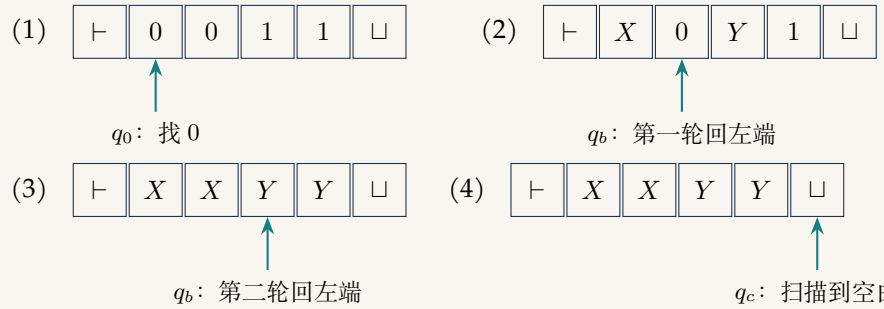
例 3.2　判定 0^n1^n

使用标记 X, Y 分别表示已配对的 $0, 1$。每一轮，从左端寻找第一个未标记的 0，改为 X；继续向右寻找一个未标记的 1，改为 Y；回到左端，开始下一轮。
如果寻找 1 时先遇到空白，拒绝。若寻找 0 时先遇到 Y，说明 0 已全部处理，此后必须全为 Y，直到空白才能接受。若在寻找首个未处理 0 时直接遇到 1，拒绝。空串接受。
输入 0011 在每轮结束后变为

$$0011 \longrightarrow X0Y1 \longrightarrow XXYY,$$

最后确认只剩 X 与 Y 后接受。输入 001 在第二轮找不到新 1；输入 0101 在已进入 Y 区域后又看见 0，因此都拒绝。

例 3.2　判定 0^n1^n（续）



输入 0011 的四个关键格局。最后一幅中再执行一步空白转移，就进入接受状态。

这台机器可以明确写成下表。表项为“新状态，写入符号，移动方向”；acc、rej 代表停机状态。输入字母表为 $\{0,1\}$，带字母表增加 $X, Y, \sqcup, \vdash$。未列出的情况进入拒绝状态。

所读符号	q_0 找 0	q_1 找 1	q_b 回左端	q_c 检查尾部
0	(q_1, X, R)	$(q_1, 0, R)$	$(q_b, 0, L)$	rej
1	rej	(q_b, Y, L)	$(q_b, 1, L)$	rej
X	(q_0, X, R)	rej	(q_b, X, L)	rej
Y	(q_c, Y, R)	(q_1, Y, R)	(q_b, Y, L)	(q_c, Y, R)
$\sqcup$	acc	rej	rej	acc
$\vdash$	rej	rej	$(q_0, \vdash, R)$	rej

在每一轮成功配对后，未标记的 0 数量减少一个。每次扫描都在有限输入区间内结束，因此所有输入上都停机。正确性由每轮恰好消去一对 $0,1$，以及对先后顺序的检查共同保证。

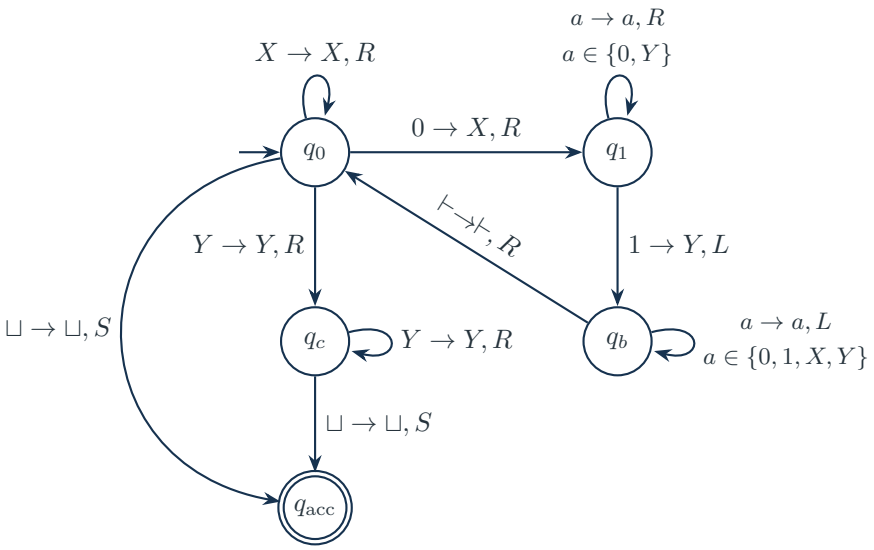


图 3.1　判定 0^n1^n 的 TM。边标签依次写出所读符号、写入符号和移动方向。没有画出的情况全部进入拒绝状态，与转移表一致。

图 3.1把同一张转移表画成状态图。中间的回边完成一轮配对，左侧的 q_c 负责最后的尾部检查。

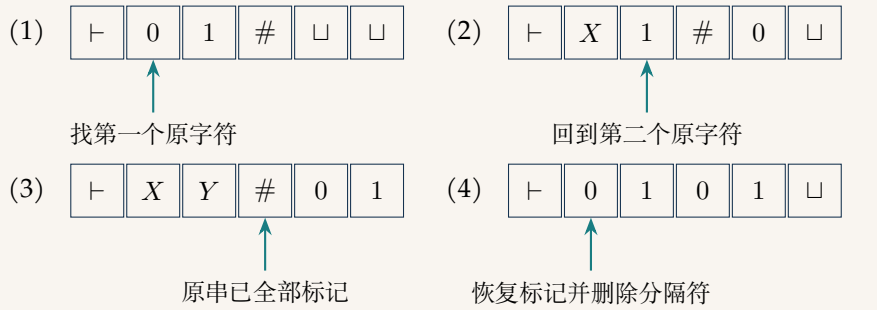
例 3.3　复制一个串

要把 w 变为 ww，先在 w 后写一个分隔符 $\#$，形成 $w\#$。反复从左端寻找分隔符之前第一个未标记字符，用 X, Y 区分原来的 $0, 1$，并将该字符复制到最右端的空白格。每次追加后回到左端。

例如 $w = 01$ 时，

$$01\# \longrightarrow X1\#0 \longrightarrow XY\#01.$$

全部原字符标记完后，将 X, Y 恢复为 $0, 1$，再删除分隔符并把其右侧内容左移一格，得到 0101。分隔符明确划分原串与复制区，避免把新写入的字符再次当作待复制字符。



$\#$ 左侧是原串，右侧是复制区。X, Y 记录原字符已经复制，最后再统一恢复。

例 3.4　判定三个数量相等

TM 可以判定 $\{0^n1^n2^n : n \geq 0\}$。先检查输入属于 $0^*1^*2^*$；随后每轮依次标记一个 0、一个 1、一个 2。任何一类提前耗尽就拒绝；三类同时耗尽就接受。

输入 001122 的轮次为 $001122 \to X0Y1Z2 \to XXYYZZ$。每轮减少三个未标记字符，因此会停机。第二章证明了该语言不是 CFL；这个算法展示了 TM 如何用多次扫描完成 PDA 难以同时维持的计数匹配。

3.3　转换器、判定器、识别器与枚举器

同一 TM 模型可以承担不同任务：

工作方式	输出含义	停机要求
转换器	输入 x，输出一个串 $f(x)$	计算全函数时处处停机；计算部分函数时允许无输出地运行
判定器	对输入作出接受或拒绝的正确判断	每个输入上都停机
识别器	恰好接受属于语言的输入	非成员可以拒绝，也可以永不停机
枚举器	从空白输入开始，依次打印语言中的串	可持续运行，允许重复和任意输出顺序

识别器 M 的语言仍定义为 $L(M) = \{x : M \text{ 接受} x\}$。停机拒绝与不停机都表示输入不属于 $L(M)$，但两种运行行为不同。

例 3.5　同一个语言的两台机器

令 $L=\{0\}$。机器 D 比较输入是否恰为 0，相等则接受，否则拒绝；它是判定器。机器 R 也在输入为 0 时接受，但其他输入上进入永远向右移动的循环；它是识别器。
两台机器接受的语言相同，停止行为不同。因此“某台机器会循环”本身并不说明它接受的语言不可判定。

3.4　格局与计算历史

格局记录机器在某一时刻的全部运行信息：状态、带内容和读写头位置。若读写头正扫描 v 的第一个字符，常将格局写作 uqv。有限表示中省略右端连续空白；需要扫描新空白时再显式补出。

用 $C\vdash_M C'$ 表示按 M 的一条转移可以从 C 走到 C'。这里 $\vdash_M$ 读作“经 M 一步得到”，是连接两个格局的关系符号；写在纸带串最左端的 $\vdash$ 则仍是左端标记。二者根据出现位置区分。例如 $\vdash 1q001\ \vdash_M \vdash 11r01$ 中，两端串开头的符号标记纸带边界，中间带下标的符号表示一步转移。一般地，若 $\delta(q,a)=(r,b,R)$ 时，

$$uqav\ \vdash_M\ ubrv.$$

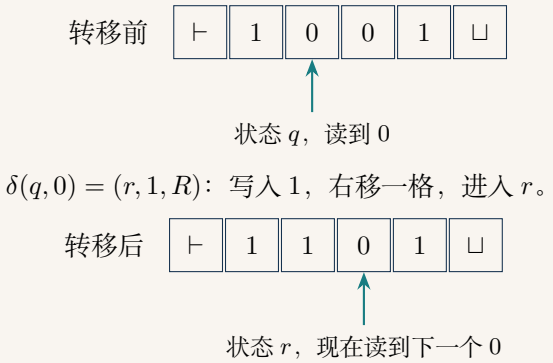
若 $\delta(q,a)=(r,b,L)$ 且 $u=u'c$ 非空，则

$$u'cqav\ \vdash_M\ u'rcbv.$$

两式说明一步转移只改变读写头附近的有限位置。

例 3.6　直接核对一步转移

若 $\delta(q,0)=(r,1,R)$，格局 $\vdash 1q001$ 变为 $\vdash 11r01$。状态符号右边原来的 0 被写成 1，状态符号右移到下一格之前。核对这一步只需要查询一条有限转移规则并比较其余带内容。



有限序列 $C_0,C_1,\dots,C_t$ 称为计算历史，若 C_0 是给定输入的初始格局，且每对相邻格局都满足一步转移关系。若末格局处于接受状态，就是接受计算历史。可用分隔符把它编码成一个有限串。

给定 M,w 和一个候选历史 h，总能在有限时间内检查：格式是否正确、首格局是否对应 w、末格局是否接受、相邻格局是否正确。因此“h 是 M 在 w 上的接受历史”是可判定的关系。寻找这样一个历史可能一直没有结果；验证已经给出的有限历史则总能结束。

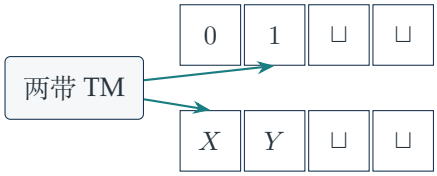
3.5 TM 的变体

多带 TM

k 带 TM 有 k 条带和 k 个独立读写头，转移函数一次读取 k 个符号：

$$\delta : Q \times \Gamma^k \to Q \times \Gamma^k \times \{L, R, S\}^k.$$

单带机可以用分隔符依次存放每条虚拟带，并用带标记的字符表示各虚拟读写头。模拟一步时，先扫描所有段，记下各读写头所读符号；再扫描一遍完成写入和头标记移动。需要延长某段时，移动后面的有限内容以腾出空间。



单带表示：$\#\,0\,\dot{1}\,\sqcup\,\#\,\dot{X}\,Y\,\sqcup\,\#$
点号标出虚拟读写头位置。

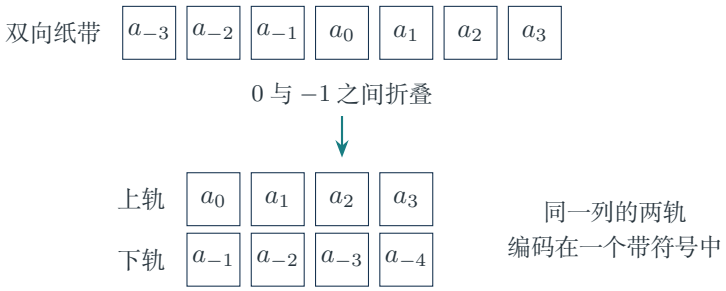
模拟会增加运行时间，但每个模拟步骤仍然有限。反方向只使用第一条带即可，所以两种模型在可识别性和可判定性上等价。

例 3.7　两带机复制输入

第一条带保存 w，第二条初始为空。两个头同步右移，把第一条带读到的字符写到第二条。扫描到第一条带的空白后停止，就得到两份 w。这比单带上的反复往返容易描述；等价模拟保证这种写法仍对应普通 TM 算法。

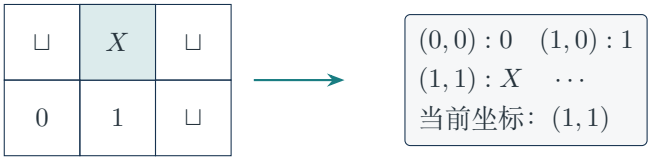
双向无限带、多维带与多头

双向无限带的格子编号为整数。可把非负半轴和负半轴折叠为两条轨道：位置 $0, 1, 2, \ldots$ 放上轨，$-1, -2, -3, \ldots$ 放下轨，有限控制记录当前轨道及跨越原点的移动。



下轨的坐标按 $-1, -2, -3, \ldots$ 排列。有限控制记录轨道，并处理跨越原点的移动。

多维带可以用有限记录表保存所有已经访问的坐标和内容，例如二维位置用整数对 (i, j) 编码。每次移动先更新坐标，再搜索相应记录；不存在的记录视为空白并在需要时添加。任一有限时刻只访问有限多个格子，因此普通 TM 能完成每一步。



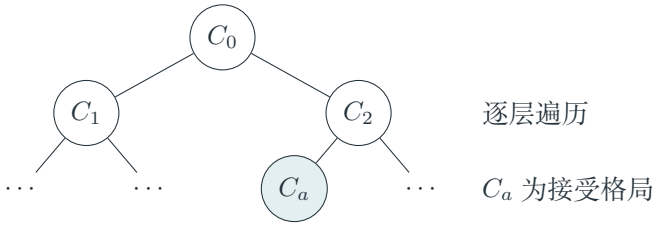
二维带的已访问区域

单带机把坐标和内容存成有限记录表。更新读写头时先改变坐标，再查找或新建记录。

多头 TM 仍可在带上给每个虚拟头设置不同标记，分多次扫描收集符号并更新。若多个头同时写同一格，先固定模型规定的冲突处理规则，再按该规则模拟。

非确定性图灵机 NTM

NTM 的转移给出有限多个候选动作。输入被接受，表示存在一条有限接受路径。确定性 TM 可以按广度优先次序遍历计算树：先检查深度 0，再检查深度 $1, 2, \ldots$。有限分支保证每层有限，因此任何有限接受路径最终都会被发现。



深度优先遍历可能困在一条无限分支上，广度优先则保证所有有限路径都得到检查。

若 NTM 对每个输入的每条分支都停机，则它是非确定性判定器。固定输入下，其计算树有限分支且没有无限路径，因而整棵树有限；否则可以不断选择有无限多个后代的子节点，构造一条无限路径。确定性模拟在遍历完全部分支且没有接受时拒绝，故也处处停机。

3.6 丘奇–图灵论题

丘奇–图灵论题将直观上的有效计算与 TM 可计算性联系起来：任何能由有限、机械、明确步骤执行的计算，都可以由图灵机实现。它把一个非形式化概念与数学模型相对应，因此称为论题。

多带机、NTM、λ 演算和递归函数等形式模型之间的等价性，则是可以在各自定义下证明的数学结论。这些结果解释了为什么选择具体机器形式通常不影响可计算性结论。效率问题还要另外分析：一个模拟存在，并不意味着它与原机器耗时相同。

第 4 章　不可判定性

一台具体机器是否停止，与一个语言是否存在判定算法，是不同层次的问题。本章先建立可识别、可判定和补可识别之间的关系，再用编码、通用模拟与对角化证明一般的接受问题不可判定。

4.1　可识别、可判定与补可识别

若存在识别器接受语言 L，则称 L 为图灵可识别语言，所有这类语言组成 RE。若存在对每个输入都停机的判定器，则 L 可判定，记 $L \in \mathrm{REC}$。若补语言 $\overline{L}$ 可识别，则称 L 为补图灵可识别语言，记 $L \in \mathrm{coRE}$。

也可以用一台“遇到非成员必停机拒绝、遇到成员允许接受或不停机”的机器描述补可识别性。将补语言识别器的接受与拒绝状态交换，就得到这种行为。可判定语言的补仍可判定，因此“补可判定”与“可判定”是同一类。

可判定性的双侧识别刻画

$$L \in \mathrm{REC} \iff L \in \mathrm{RE} \text{ 且 } \overline{L} \in \mathrm{RE}. \qquad \text{即} \qquad \mathrm{REC} = \mathrm{RE} \cap \mathrm{coRE}.$$

证明。若有判定器，交换其接受和拒绝状态即可判定补语言，两者当然都可识别。反过来，设 M_1 识别 L、M_2 识别 $\overline{L}$。在输入 w 上交替模拟两台机器各一步。M_1 接受则接受，M_2 接受则拒绝。每个 w 恰属于其中一个语言，所以相应识别器最终会接受，整个过程必停机。 □

例 4.1　交替模拟的必要性

假设 $w \notin L$，$M_1(w)$ 恰好永不停止，而 $M_2(w)$ 在 200 步后接受。如果先完整运行 M_1，就永远没有机会运行 M_2。交替运行时，M_2 仍能在有限时间内完成这 200 步，并给出拒绝 w 的结论。模拟调度必须保证每个仍在运行的任务都持续前进。

4.2　枚举与识别

枚举定理

语言图灵可识别，当且仅当存在一个枚举器，其打印出的串恰好组成该语言。

证明。若有枚举器 E，识别输入 w 时运行 E，每打印一个串就与 w 比较，遇到相同串便接受。w 属于语言时一定会等到；不属于时允许一直等待。

反过来，设 M 识别 L，将全部串按标准序列为 $s_1, s_2, \ldots$。第 t 轮分别模拟 $M(s_1), \ldots, M(s_t)$ 的前 t 步，打印其中已经接受的输入。每轮是有限计算。若 $M(s_j)$ 在 r 步接受，第 $\max(j, r)$ 轮就能发现它。未接受的输入永不打印。这称为交错模拟或鸽尾式模拟（dovetailing）。 □

例 4.2　枚举过程不会被某个输入卡住

假设 $M(s_1)$ 永不停止，$M(s_2)$ 在第三步接受，$M(s_3)$ 在第一步接受。第三轮会检查前三个输入各三步，因此打印 s_2, s_3；s_1 的运行并不阻止它们出现。若语言为空，则枚举器持续计算而不打印任何串，这也符合定义。



第 t 轮检查前 t 个输入的前 t 步，每个固定计算最终都会得到足够多的模拟时间。

按标准序枚举

若一个无限语言存在严格按标准序、无重复输出的枚举器，就可以判定它：等待输出等于输入 w，或输出已经大于 w 的串。前者接受，后者拒绝。由于语言无限，输出长度不可能有界，最终总能超过任何给定串。有限语言本身也可判定，但一般枚举器未必会通知“已经输出完毕”，所以不能直接把等待下一个输出当成终止条件。

4.3　编码与通用图灵机

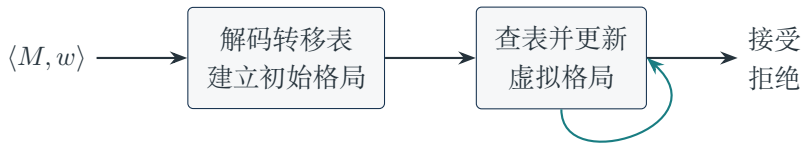
有限的状态集、字母表和转移表都可以写成有限字符串。用 $\langle M \rangle$ 表示机器编码，用 $\langle M, w \rangle$ 表示机器与输入的联合编码。采用一种能够有效编码、解码并检查格式的固定方案即可；理论结论不依赖某一种具体字符布局。

例 4.3　为有限对象设计编码

可以先给状态和带符号编号，把一条转移写成五个自然数 (i, j, k, ℓ, d)，再用分隔符或自定界编码连接。一个简单的自然数自定界编码是 1^n0：先数连续的 1，遇到 0 表示该数结束。多个数的编码可以直接拼接并唯一分割。

这样的编码效率不高，但格式检查、拼接、修改和读回都能由 TM 完成。归约中“生成一台新机器的代码”正是这类有限字符串操作。

存在通用图灵机 U：输入 $\langle M, w \rangle$ 后，解码 M 的转移表和 w，维护一份虚拟格局，按表逐步更新，模拟 $M(w)$。若 M 接受或拒绝，U 给出相应结果；若 M 一直运行，U 也一直模拟。



循环箭头表示模拟下一步；通用模拟能够忠实运行机器，不会自动提前判断该循环何时结束。

下文所有机器语言都先要求输入是合法编码；不合法编码为非成员。补语言相对于整个编码字母表上的串取补，因此包含不合法编码。构造归约时可先检查格式，再将异常格式映到真假相符的固定实例。

4.4 接受问题与对角化

定义图灵机接受语言

$$A_{\mathrm{TM}} = \{\langle M, w\rangle : M \text{ 接受} w\}.$$

通用模拟给出其识别器：模拟 $M(w)$，见到接受就接受；见到拒绝就拒绝；模拟不结束时保持运行。因此 $A_{\mathrm{TM}} \in \mathrm{RE}$。

A_{TM} 不可判定

不存在对所有 $\langle M, w\rangle$ 都停机，并正确判断 M 是否接受 w 的 TM。

证明。假设有判定器 H。构造机器 D，输入 $\langle M\rangle$ 后运行 $H(\langle M, \langle M\rangle\rangle)$，把结果反过来：$H$ 接受则 D 拒绝，H 拒绝则 D 接受。H 处处停机，所以 D 也处处停机。

现在将 D 自己的编码输入 D。如果 H 判断 D 接受 $\langle D\rangle$，按 D 的定义它却拒绝；如果 H 判断 D 不接受 $\langle D\rangle$，它却接受。两种情况都矛盾，所以 H 不存在。 □

例 4.4　对角线上的结果翻转

把机器 $M_1, M_2, \ldots$ 排成行，把编码 $\langle M_1\rangle, \langle M_2\rangle, \ldots$ 排成列。若假设 H 存在，表中每个格子都能在有限时间内填写“接受”或“不接受”。D 在第 i 个输入上，给出第 i 行、第 i 列结果的反值。

因此 D 的接受行为与第 i 台机器至少在第 i 列不同。但 D 本身也应当出现在某一行，这一行的对角位置就要求一个结果等于自身的反值。

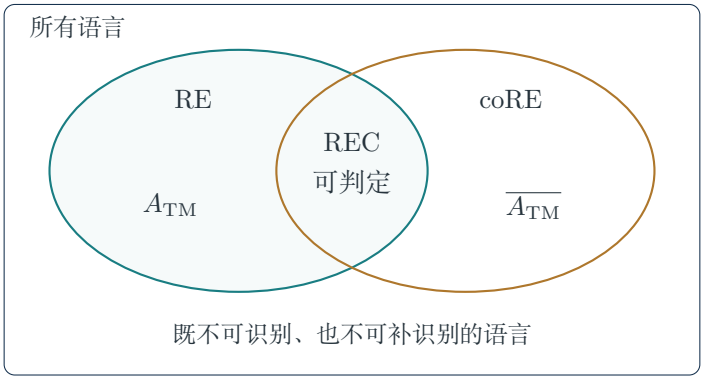
	$\langle M_1\rangle$	$\langle M_2\rangle$	$\langle M_3\rangle$
M_1	b_1	·	·
M_2	·	b_2	·
M_3	·	·	b_3
D	$1-b_1$	$1-b_2$	$1-b_3$

b_i 是假设的判定器给出的第 i 个对角答案；1 表示接受，0 表示不接受。

D 与每一行至少在对应的对角位置不同，因此它无法成为表中的任何一行。

$\overline{A_{\mathrm{TM}}}$ 不可识别

由于 A_{TM} 可识别但不可判定，$\overline{A_{\mathrm{TM}}}$ 不可识别；否则两侧识别刻画会使 A_{TM} 可判定。因此 $\overline{A_{\mathrm{TM}}} \in \mathrm{coRE} \setminus \mathrm{RE}$。



RE 与 coRE 的交集是 REC。两侧都能确认成员资格时，交替模拟就能得到判定器。

从数量看不可计算性

每台 TM 都有有限编码，所以全部 TM 可数。只要字母表非空，Σ^* 是可数无限集，而它的全部子集组成的语言集合 $\mathcal{P}(\Sigma^*)$ 不可数。因此有些语言连识别器都没有。这个计数论证说明不可识别语言存在；对角化则给出了具体、可研究的语言。

4.5 可判定语言的封闭性

设 D_A, D_B 分别判定 A, B。

- **并、交**。在输入 w 上运行两台判定器，再对结果取“或”或“且”。两台都停机，所以组合也停机。
- **补**。交换 D_A 的接受与拒绝结果。
- **连接**。对 w 的全部 $|w|+1$ 个切分 $w = uv$，检查 $u \in A$、$v \in B$，存在一次两者均真便接受；全部失败则拒绝。
- **星**。空串直接接受。非空串只需考虑将它切成非空段的所有方式，共有 $2^{|w|-1}$ 种。若某个切分的每一段都属于 A，接受，否则拒绝。含空段的连接可删除空段，不影响非空串的表示。
- **反转**。先算出 w^R，再运行 $D_A(w^R)$。

连接和星运算虽然可能需要很多次计算，但对固定输入只有有限种切分，每个子计算又会停机，因此构造的是判定器。

例 4.5 检查一个连接语言

设 $A = \{0^n 1^n : n \geq 0\}$、$B = \{1^m 2^m : m \geq 0\}$。判定 $001112 \in AB$ 时，可以逐一尝试所有切分，其中

$$001112 = (0011)(12)$$

两段分别属于 A, B，因此接受。机器只需找到一个成功切分；对于没有成功切分的输入，有限枚举最终也会结束。

4.6 可识别与补可识别语言的封闭性

可识别语言

对并集，同时模拟两个识别器，有一个接受就接受；对交集，等到两个都接受再接受。连接时，同时尝试全部切分，为每个切分模拟两侧识别器，某个切分的两个子任务都接受就接受。星运算同理：同时模拟所有非空分段方案，某个方案的每一段均被接受就接受。反转只需先反转输入。

每个成员都有一个有限成功见证：一个接受任务、一对接受任务，或一个有限分段上的全部接受任务。交错模拟保证这种成功会被找到。可识别类对补不封闭，A_{TM} 与 $\overline{A_{\mathrm{TM}}}$ 已给出反例。

补可识别语言

对并与交，使用德摩根公式和 RE 对并、交的封闭性即可。反转满足 $\overline{A}^R = (\overline{A})^R$，因此也保持补可识别性。

连接与星需要直接利用“固定输入只有有限种切分”。若 $A, B \in \mathrm{coRE}$，我们识别 $\overline{AB}$。对每个切分 $w = uv$，交替运行 $\overline{A}$ 对 u 和 $\overline{B}$ 对 v 的识别器；只要其中一个接受，就证明这个切分失败。当全部切分均已有失败证据时，接受 w。

若 $w \notin AB$，每个切分确实至少一侧失败，有限多个证据最终都会出现；若 $w \in AB$，某个成功切分永远不会得到失败证据，因而整体不会错误接受。星运算也按同样方式，为每个非空分段方案等待至少一个不属于 A 的段，全部方案均失败才接受。ε 总属于 A^*，所以在识别其补时直接拒绝 ε。

例 4.6　补可识别性的有限“全部失败”检查

长度为三的串 abc 有四个二段切分：

$$(\varepsilon, abc), \quad (a, bc), \quad (ab, c), \quad (abc, \varepsilon).$$

要证明 $abc \notin AB$，必须为这四项各找到一个失败侧。它们的完成时间可以不同；先记住已完成的项，其余项继续模拟。当第四项也得到证据时，机器即可停机接受“非成员”这一结论。这依赖切分数有限。若把问题改成“对所有输入串都失败”，输入串有无限多个，上述等待方式就不再提供有限停机依据。

第 5 章　归约

已经知道一个问题不可判定后，可以把它的实例转换成另一个问题的实例，从而证明后者也不可判定。归约的核心是一个能够实际执行的转换，以及转换前后答案相同的双向证明。

5.1 映射归约

称 A 可映射归约到 B，记作 $A \le_m B$，若存在一个全可计算函数 $f : \Sigma^* \to \Sigma^*$，满足

$$\forall x, \qquad x \in A \iff f(x) \in B.$$

“全”表示每个输入上都能在有限时间内生成输出。归约函数可以把输入变成数字、文法或机器编码；生成机器的代码并不要求先运行这台机器。



若右侧判定器存在，整个过程就会判定 A。

基本性质

1. **向下保持可判定性和可识别性**。若 $A \le_m B$ 且 B 可判定，则先算 $f(x)$ 再判定 B，可得 A 可判定。同样地，B 可识别或补可识别时，A 也具有相应性质。取逆否命题即可传播不可判定性或不可识别性。
2. **自反与传递**。恒等函数给出 $A \le_m A$。若 $A \le_m B$ 通过 f、$B \le_m C$ 通过 g，则复合函数 $g \circ f$ 给出 $A \le_m C$。
3. **同时取补**。同一个 f 满足 $x \in A \iff f(x) \in B$，也满足 $x \in \overline{A} \iff f(x) \in \overline{B}$。因此 $A \le_m B \iff \overline{A} \le_m \overline{B}$。

为了证明目标 B 不可判定，应把已知不可判定的 A 归约到 B。证明反方向只会说明 B 能由 A 解决，无法据此排除 B 本身很简单的可能。

例 5.1　可判定语言之间的归约

若 A 可判定，而 B 同时有一个成员 b_1 和一个非成员 b_0，则

$$f(x) = \begin{cases} b_1, & x \in A, \\ b_0, & x \notin A \end{cases}$$

是全可计算归约。因此任意两个非平凡可判定语言彼此可映射归约。这里的归约不限制运行时间，讨论的是可计算性层次。

5.2　接受、停机与空性

定义

$$\mathrm{HALT}_{\mathrm{TM}} = \{\langle M, w\rangle : M \text{ 在} w \text{ 上停机}\}.$$

例 5.2　$A_{\mathrm{TM}} \le_m \mathrm{HALT}_{\mathrm{TM}}$

给定 $\langle M, w\rangle$，生成机器 N：

1. 在任何输入上模拟 $M(w)$。
2. 若模拟接受，则停机；若模拟拒绝，则进入无限循环。

令 $f(\langle M, w\rangle) = \langle N, \varepsilon\rangle$。于是

$$M \text{ 接受} w \iff N \text{ 在} \varepsilon \text{ 上停机}.$$

若 $M(w)$ 不停机，N 也持续模拟。构造 N 时只是把 M, w 写入一段固定程序模板，故 f 总能结束。由 A_{TM} 不可判定，$\mathrm{HALT}_{\mathrm{TM}}$ 不可判定。直接模拟又说明 $\mathrm{HALT}_{\mathrm{TM}}$ 可识别。

例 5.3 $\mathrm{HALT}_{\mathrm{TM}} \le_m A_{\mathrm{TM}}$

生成机器 N：模拟 $M(w)$，只要模拟停机，无论接受还是拒绝，都接受自己的输入。则

$$\langle M, w\rangle \in \mathrm{HALT}_{\mathrm{TM}} \iff \langle N, \varepsilon\rangle \in A_{\mathrm{TM}}.$$

这说明接受问题与停机问题彼此可映射归约。两者的定义仍应区分：接受是特定的停机结果，停机包含接受与拒绝。

定义空性问题

$$E_{\mathrm{TM}} = \{\langle M\rangle : L(M) = \varnothing\}.$$

其补语言可以识别：交错运行 M 在全部输入上的计算，只要某个输入被接受，就确认语言非空。因此 $E_{\mathrm{TM}} \in \mathrm{coRE}$。

例 5.4 从接受问题到非空性

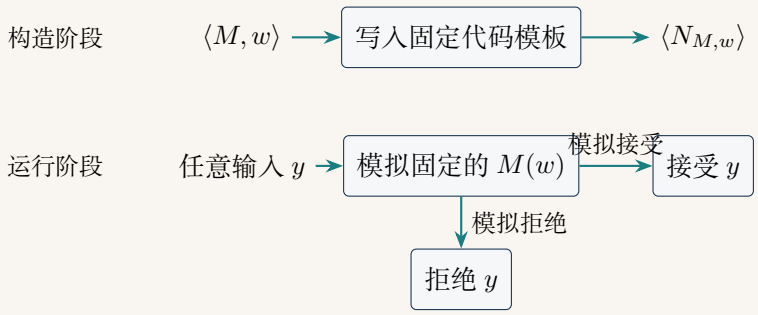
对给定 $\langle M, w\rangle$，构造 $N_{M,w}$：“输入任意 y，模拟 $M(w)$；若它接受，则接受 y。”于是

$$L(N_{M,w}) = \begin{cases} \Sigma^*, & M \text{ 接受} w, \\ \varnothing, & M \text{ 不接受} w. \end{cases}$$

故 $A_{\mathrm{TM}} \le_m \overline{E_{\mathrm{TM}}}$，同时 $\overline{A_{\mathrm{TM}}} \le_m E_{\mathrm{TM}}$。若 E_{TM} 可识别，则 $\overline{A_{\mathrm{TM}}}$ 可识别，矛盾。因此

$$E_{\mathrm{TM}} \in \mathrm{coRE} \setminus \mathrm{RE}.$$

这里“M 不接受”同时包含停机拒绝与永不停机，两种情形都不会使 N 接受任何输入。



生成机器代码时只写入参数，不等待 $M(w)$ 的运行结果。模拟发生在新机器被运行之后；若模拟持续运行，新机器也持续运行。

5.3 满性、等价与包含

定义 TM 满性问题

$$\mathrm{ALL}_{\mathrm{TM}} = \{\langle M\rangle : L(M) = \Sigma^*\}.$$

上一例的 $N_{M,w}$ 直接给出 $A_{\mathrm{TM}} \le_m \mathrm{ALL}_{\mathrm{TM}}$，所以 $\mathrm{ALL}_{\mathrm{TM}}$ 不可判定，且不属于 coRE。

例 5.5 从补接受问题到满性

给定 $\langle M, w\rangle$，构造一台总会停机的机器 $B_{M,w}$。在输入 y 上，它只模拟 $M(w)$ 的前 $|y|$ 步：若这段时间内出现接受，则拒绝 y；否则接受 y。

$$M \text{ 不接受} w \iff L(B_{M,w}) = \Sigma^*.$$

如果 M 在第 t 步接受，那么所有长度至少为 t 的串都被拒绝，故语言不满；如果永远不接受，所有有限长度测试都通过。因此 $\overline{A_{\mathrm{TM}}} \le_m \mathrm{ALL}_{\mathrm{TM}}$，进一步推出 $\mathrm{ALL}_{\mathrm{TM}} \notin \mathrm{RE}$。
两次归约合起来说明，$\mathrm{ALL}_{\mathrm{TM}}$ 既不可识别，也不可补识别。第二次构造输出的机器都是判定器；难点在于判断它对无限多个输入是否全都接受。

例 5.6 两台 TM 是否等价

定义

$$\mathrm{EQ}_{\mathrm{TM}} = \{\langle M_1, M_2\rangle : L(M_1) = L(M_2)\}.$$

固定接受全部串的机器 U_{all}，映射 $\langle M\rangle \mapsto \langle M, U_{\mathrm{all}}\rangle$ 给出 $\mathrm{ALL}_{\mathrm{TM}} \le_m \mathrm{EQ}_{\mathrm{TM}}$。因此等价问题既不可识别，也不可补识别。
类似地，语言包含问题 $\mathrm{SUB}_{\mathrm{TM}} = \{\langle M_1, M_2\rangle : L(M_1) \subseteq L(M_2)\}$ 接收映射 $\langle M\rangle \mapsto \langle U_{\mathrm{all}}, M\rangle$，也有相同的两侧不可识别结论。

5.4 CFG 的成员资格与空性

定义

$$A_{\mathrm{CFG}} = \{\langle G, w\rangle : w \in L(G)\}, \qquad E_{\mathrm{CFG}} = \{\langle G\rangle : L(G) = \varnothing\}.$$

两者都可判定，但方法不同。

成员资格：CYK 算法

先将 G 转成 CNF。若输入为空，检查起始变量是否有空产生式。对非空串 $w = a_1 \cdots a_n$，令 $T[i, j]$ 为能够生成子串 $a_i \cdots a_j$ 的所有变量。初始化

$$T[i, i] = \{A : A \to a_i \in R\},$$

再按区间长度从小到大计算

$$T[i, j] = \bigcup_{k=i}^{j-1} \{A : \exists (A \to BC) \in R,\ B \in T[i, k],\ C \in T[k+1, j]\}.$$

最后检查 $S_0 \in T[1, n]$。CNF 分析树根部只有二叉拆分，所以对区间长度归纳即可证明表项准确。区间、切分点和产生式均有限，算法必停机。

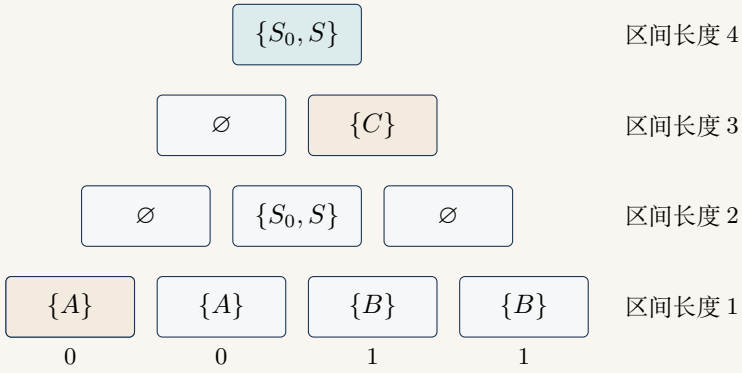
例 5.7 手算 CYK 表

考虑 CNF

$$S_0 \to AB \mid AC, \quad S \to AB \mid AC, \quad C \to SB, \quad A \to 0, \quad B \to 1.$$

例 5.7　手算 CYK 表（续）

输入 0011。单字符区间依次得到 $\{A\},\{A\},\{B\},\{B\}$。长度为二的区间中，只有 $T[2,3]=\{S_0,S\}$ 非空。长度为三时，$C\to SB$ 给出 $T[2,4]=\{C\}$。最后 $S_0\to AC$ 给出 $S_0\in T[1,4]$，所以接受。每个非空表项都能对应一棵生成该子串的分析树。



从下往上填表。最顶格利用黄色的 $\{A\}$ 与 $\{C\}$，通过 $S_0\to AC$ 接受 0011。

空性：生成能力标记

先标记能一步生成终结串的变量，包括空串规则的左侧。反复扫描产生式，如果右侧每个变量都已标记，就标记左侧。有限变量集保证过程终止。对派生树高度归纳，最终被标记的变量恰为能够生成某个终结串的变量。因此 S 未标记当且仅当语言为空。

例 5.8　一个永远无法完成的派生

设

$$S\to AB,\qquad A\to aA\mid a,\qquad B\to C,\qquad C\to B.$$

A 能生成 a，所以被标记；B,C 只在彼此之间替换，没有终结出口，因此不会被标记。S 的右侧需要 B，也无法标记，故 $L(G)=\varnothing$。拥有无限多条变量替换路径，并不意味着能生成终结串。

5.5　计算历史与 CFG 满性

定义

$$\mathrm{ALL}_{\mathrm{CFG}}=\{\langle G\rangle : L(G)=\Sigma^*\}.$$

由于成员资格可判定，可以枚举所有串，寻找一个不属于 $L(G)$ 的串。因此 $\overline{\mathrm{ALL}_{\mathrm{CFG}}}$ 可识别，满性问题属于 coRE。

为证明它不可判定，给定 M,w，构造一个 CFG，生成所有不是 $M(w)$ 的合法接受历史的串。关键在于把历史写成交替反转的形式：

$$C_0\#C_1^R\#C_2\#C_3^R\#\cdots.$$

各格局统一补足右侧空白到某个共同长度，这个长度可以随历史而变。只要 $M(w)$ 有有限接受计算，就存在足够长的这种编码。

若要统一使用二进制字母表，可为当前机器的状态、带符号和分隔符选择固定长度的二进制块编码。块长和码表可由机器描述有效算出，有限控制即可识别。将所有格式不合法的二进制串也归入错误历史语言；这样，无论历史使用什么内部符号，满性始终相对于固定的 $\{0,1\}^*$。

候选串不合法，可以由以下一种错误证明：格式错误，初始格局错误，末格局不接受，或某对相邻格局不满足一步转移。前三类可以由有限状态检查。最后一类由 PDA 非确定地选择相邻两段：把第一段压栈，再读取第二段并弹栈比较。由于第二段反向编码，对应带格会依次相遇。



绝大多数位置应保持相同；
读写头附近只允许转移表规定的局部变化。

一步 TM 转移只改变局部有限位置，因此有限控制可以记住所需的小窗口；栈负责对齐其余任意长的部分。对于另一种奇偶位置，交换读取方向即可。由有限次并构造，全部错误编码组成 CFL，且相应 PDA、CFG 都能由 M, w 有效生成。

例 5.9　由错误历史构造满性归约

将上面得到的文法记为 $G_{M,w}$。若 M 不接受 w，根本不存在合法接受历史，所以每个串都是错误编码，$L(G_{M,w}) = \Sigma^*$。若 M 接受 w，至少有一个合法历史不在该语言中，所以 $L(G_{M,w}) \neq \Sigma^*$。因此

$$\overline{A_{\mathrm{TM}}} \leq_m \mathrm{ALL}_{\mathrm{CFG}}.$$

由此，$\mathrm{ALL}_{\mathrm{CFG}}$ 不可识别，更不可判定；结合前面的上界，它恰好属于 $\mathrm{coRE} \setminus \mathrm{RE}$。

例 5.10　CFG 等价与包含

固定生成 Σ^* 的文法 G_{all}。由

$$L(G) = \Sigma^* \iff L(G) = L(G_{\mathrm{all}})$$

可知 $\mathrm{ALL}_{\mathrm{CFG}} \leq_m \mathrm{EQ}_{\mathrm{CFG}}$。另一方面，若两个 CFG 不等价，可以枚举一个恰好属于其中一个语言的串，并用成员资格算法确认，所以等价问题属于 coRE，且不可判定。
对包含问题 $\mathrm{SUB}_{\mathrm{CFG}}$，寻找 $L(G_1) \setminus L(G_2)$ 中的串可识别其补；映射 $G \mapsto (G_{\mathrm{all}}, G)$ 证明不可判定。两类问题都属于 $\mathrm{coRE} \setminus \mathrm{RE}$。

5.6　LBA 的成员资格、空性与满性

线性界限自动机（LBA）是工作带使用范围受输入长度线性约束的 TM，可允许非确定性。固定模型可以只在输入的两个端标记之间工作；常数倍空间可通过扩大带字母表、使用多轨道编码吸收。空输入单独处理。

成员资格可判定

对输入长 n、最多使用 $cn + c$ 个格子的 LBA，可能格局数有一个可计算的有限上界，例如

$$|Q|(cn + c)|\Gamma|^{cn+c}$$

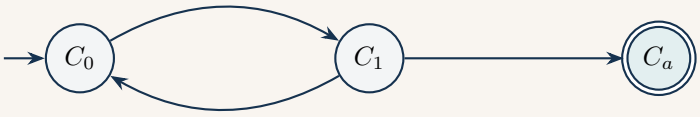
乘上固定端标记约定产生的常数因子。将格局作为有限有向图的顶点，合法一步计算作为边，就能从初始格局做可达性搜索。找到接受格局则接受，全部可达格局搜索完而没有接受则拒绝。重复格局无需继续展开。因此

$$A_{\mathrm{LBA}} = \{\langle B, w\rangle : B \text{ 接受} w\}$$

可判定，即使某些计算分支会无限循环。

例 5.11　有限格局图中的循环

假设从初始格局 C_0 可以到 C_1，再回到 C_0，同时 C_1 还有另一条边到接受格局 C_a。直接沿第一条路径运行可能一直循环；图搜索记录已访问顶点，仍会检查 $C_1 \to C_a$ 并接受。若所有可达节点只有 C_0, C_1，搜索也能在有限时间内拒绝。



记录已访问的 C_0, C_1，
继续检查通向 C_a 的边。

空性不可判定

给定 M, w，构造 LBA $B_{M,w}$，输入是一个候选有限接受历史。它直接核对格式、首末格局和相邻转移，恰好在输入是合法历史时接受。可以保留输入，利用轨道上的标记多次往返比较相邻格局，额外存储始终不超过输入长度的常数倍。因此它确实是 LBA，并且可以设计为在每个输入上停机。

例 5.12　从接受问题到 LBA 非空性

若 M 接受 w，它有一份有限接受历史，故 $L(B_{M,w}) \neq \varnothing$；若不接受，就没有合法接受历史，故语言为空。于是

$$A_{\mathrm{TM}} \leq_m \overline{E_{\mathrm{LBA}}}, \qquad \overline{A_{\mathrm{TM}}} \leq_m E_{\mathrm{LBA}}.$$

非空性可通过枚举输入并使用 A_{LBA} 的判定算法识别，所以 $E_{\mathrm{LBA}} \in \mathrm{coRE} \setminus \mathrm{RE}$。固定一个输入时格局图有限；空性则同时询问所有长度的输入，这两个问题的范围不同。

满性不可判定

对于刚才构造的、处处停机的历史检查器，交换接受和拒绝，得到错误历史检查器 $C_{M,w}$，仍是确定性 LBA。

$$M \text{ 不接受} w \iff L(C_{M,w}) = \Sigma^*.$$

因此 $\overline{A_{\mathrm{TM}}} \leq_m \mathrm{ALL}_{\mathrm{LBA}}$。又因为 LBA 成员资格可判定，枚举一个不被接受的串可以识别满性问题的补，所以 $\mathrm{ALL}_{\mathrm{LBA}} \in \mathrm{coRE} \setminus \mathrm{RE}$。

5.7　更多语言性质与 Rice 定理

许多问题只询问 $L(M)$ 的性质，而不关心机器的状态名称、运行时间或代码写法。Rice 定理统一处理这种情况。

Rice 定理

设 $\mathcal{P}$ 是可识别语言的一种非平凡性质：有可识别语言满足它，也有可识别语言不满足它。则

$$I_{\mathcal{P}} = \{\langle M\rangle : L(M) \in \mathcal{P}\}$$

不可判定。

证明。先设 $\varnothing \notin \mathcal{P}$，固定一个满足性质的可识别语言 L_0 及识别器 M_0。给定 $\langle M, w\rangle$，构造 N：输入 y 后先模拟 $M(w)$，只有看到接受才转而运行 $M_0(y)$。于是 M 接受 w 时 $L(N) = L_0$，否则 $L(N) = \varnothing$。因此 $A_{\mathrm{TM}} \leq_m I_{\mathcal{P}}$。

如果 $\varnothing \in \mathcal{P}$，就在可识别语言范围内考虑相反性质。相反性质非平凡且不包含空语言，由前一种情况，其索引问题不可判定。若原性质可判定，检查合法编码并将结果取反便能判定相反性质，矛盾。 □

例 5.13　是否接受某个固定串

固定串 u，性质 “$u \in L$” 是非平凡的语言性质，所以 $\{\langle M\rangle : u \in L(M)\}$ 不可判定。它仍可识别：直接模拟 $M(u)$。把目标输入固定为一个串，并没有使所有机器的接受行为都能提前判定。

例 5.14　语言是否有限、正则或上下文无关

下列性质都是非平凡的：

$$L(M) \text{ 有限}, \qquad L(M) \text{ 正则}, \qquad L(M) \text{ 是 CFL}.$$

例如 $\varnothing$ 同时满足三者；Σ^* 是无限正则语言；$\{0^n1^n\}$ 是非正则 CFL；$\{0^n1^n2^n\}$ 可判定但不是 CFL。由 Rice 定理，这三个 TM 语言性质问题都不可判定。
这些例子所问的是 “输入为任意 TM 时，它接受的语言是否具有该性质”。若输入已经是一台 DFA，则它接受的语言当然正则。

例 5.15　状态数属于机器描述的性质

“机器恰有十个状态” 可直接解析编码并数数，所以可判定。两个接受同一语言的机器可以有不同状态数，因此它不属于 Rice 定理讨论的语言性质。同样，Rice 定理只直接给出不可判定性；若还要判断可识别或补可识别，需要另作证明。

固定语言、交集空性与总停机

例 5.16　是否恰好接受一个指定语言

固定一个可识别语言 L_0，则 “$L(M) = L_0$” 是非平凡语言性质，故不可判定。特别地，是否恰好接受 $\{0\}$、是否只接受空串，都不可判定。

例 5.17　两台 TM 接受语言是否不相交

定义

$$\mathrm{DISJ}_{\mathrm{TM}} = \{\langle M, N\rangle : L(M) \cap L(N) = \varnothing\}.$$

它的补可识别：交错枚举串和两台机器的运行，发现某串被两者接受便停止。令 $N = U_{\mathrm{all}}$，则 $L(M) \cap L(N) = \varnothing \iff L(M) = \varnothing$，所以 $E_{\mathrm{TM}} \le_m \mathrm{DISJ}_{\mathrm{TM}}$。因此该问题属于 $\mathrm{coRE} \setminus \mathrm{RE}$。

例 5.18　是否在全部输入上停机

定义

$$\mathrm{TOT}_{\mathrm{TM}} = \{\langle M\rangle : \forall x,\ M(x) \text{ 停机}\}.$$

从 $\mathrm{ALL}_{\mathrm{TM}}$ 归约：构造 N，输入 x 后模拟 $M(x)$，只有看到接受才停机；若看到拒绝就转入无限循环。于是 N 处处停机当且仅当 M 接受全部输入。
反方向，从 $\mathrm{TOT}_{\mathrm{TM}}$ 归约到 $\mathrm{ALL}_{\mathrm{TM}}$：构造 N，模拟 $M(x)$，只要模拟停机就接受。两者彼此可归约，因此总停机问题也既不可识别、又不可补识别。

5.8　Kleene 谱系

可识别性对应“存在一个能够有限验证的见证”。当定义需要交替使用“存在”和“任意”时，就得到更高的层次。以下所有量词均遍历自然数或其有限串编码，矩阵中的关系 R 都可判定，并且没有多项式时间限制。

量词定义

对 $n \ge 1$，Σ_n 由可写成以存在量词开头、含 n 个交替量词块的语言组成；Π_n 以全称量词开头。例如

$$\begin{aligned}
\Sigma_1 &: \quad x \in L \iff \exists y\, R(x, y),\\
\Pi_1 &: \quad x \in L \iff \forall y\, R(x, y),\\
\Sigma_2 &: \quad x \in L \iff \exists y\, \forall z\, R(x, y, z),\\
\Pi_2 &: \quad x \in L \iff \forall y\, \exists z\, R(x, y, z).
\end{aligned}$$

每行都表示：存在一个可判定关系 R，使所写等价式对所有输入 x 成立。连续多个同类量词可通过元组编码合成一个量词块。定义 $\Delta_n = \Sigma_n \cap \Pi_n$。

若 L 可识别，“h 是接受历史”给出一个可判定关系，因而 $L \in \Sigma_1$；反过来，枚举 y 并检查 $R(x, y)$ 就能识别存在式。取补得到

$$\Sigma_1 = \mathrm{RE}, \qquad \Pi_1 = \mathrm{coRE}, \qquad \Delta_1 = \mathrm{REC}.$$

例 5.19　同类量词合并

空性可以写作

$$\langle M\rangle \in E_{\mathrm{TM}} \iff \forall x\, \forall t\, \neg\, \mathrm{Acc}(M, x, t),$$

其中 $\mathrm{Acc}(M, x, t)$ 表示 M 在前 t 步内接受 x，通过有限模拟可判定。把 (x, t) 编成一个数 z，就得到一个全称量词，所以 $E_{\mathrm{TM}} \in \Pi_1$。这里两个连续全称量词不会增加交替层数。

例 5.20　TM 与 CFG 满性为什么层数不同

对于 TM，

$$\langle M\rangle \in \mathrm{ALL}_{\mathrm{TM}} \iff \forall x\, \exists t\ \mathrm{Acc}(M,x,t),$$

所以属于 Π_2。对于 CFG，关系 $x \in L(G)$ 本身可判定，故

$$\langle G\rangle \in \mathrm{ALL}_{\mathrm{CFG}} \iff \forall x\, [x \in L(G)]$$

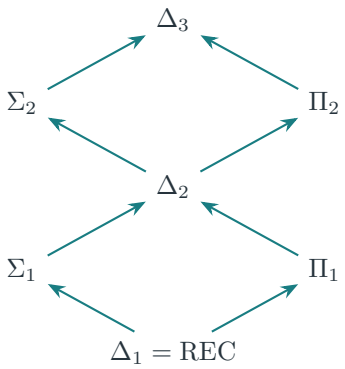
已经是 Π_1 形式。LBA 满性同理。层数取决于量词后面的关系能否直接判定，不能只看中文表述中出现了几次“所有”。

基本规则

取补会翻转每个量词，并否定可判定矩阵，因此 $L \in \Sigma_n \iff \overline{L} \in \Pi_n$。加入不参与矩阵的量词可得

$$\Sigma_n \cup \Pi_n \subseteq \Delta_{n+1}, \qquad \Delta_n \subseteq \Sigma_n, \Pi_n.$$

同一类 Σ_n 或 Π_n 对有限并、有限交封闭：把两个公式中的同类量词配对编码，再组合矩阵。它们也向下保持映射归约，因为将 x 换成全可计算的 $f(x)$ 后，矩阵仍可判定。



箭头表示语言类包含；
$\Delta_n = \Sigma_n \cap \Pi_n$。

各层的严格性可以通过相对停机问题证明，下一章介绍其具体构造。

完全语言

对语言类 $\mathcal{C}$，若 $B \in \mathcal{C}$，且每个 $A \in \mathcal{C}$ 都满足 $A \le_m B$，就称 B 为 $\mathcal{C}$ 完全语言。两个同类的完全语言必互相可归约。仅知道两个语言都属于某一层，并不足以推出互相可归约。

例 5.21　A_{TM} 是 Σ_1 完全语言

已经知道 $A_{\mathrm{TM}} \in \Sigma_1$。任取 $L \in \Sigma_1 = \mathrm{RE}$，固定其识别器 M_L。映射

$$x \longmapsto \langle M_L, x\rangle$$

总可计算，且 $x \in L \iff \langle M_L, x\rangle \in A_{\mathrm{TM}}$。所以 A_{TM} 为 Σ_1 完全语言。

由 $A_{\mathrm{TM}} \le_m \overline{E_{\mathrm{TM}}}$ 及非空性的可识别性，$\overline{E_{\mathrm{TM}}}$ 也为 Σ_1 完全语言；取补后，E_{TM} 为 Π_1 完全语言。前面已经给出的归约和上界，同样证明 $\mathrm{ALL}_{\mathrm{CFG}}$、$E_{\mathrm{LBA}}$、$\mathrm{ALL}_{\mathrm{LBA}}$ 都是 Π_1 完全语言。

例 5.22　$\mathrm{ALL}_{\mathrm{TM}}$ 是 Π_2 完全语言

任取 $L \in \Pi_2$，写成 $x \in L \iff \forall n\, \exists t\, R(x,n,t)$，其中 R 可判定。固定一个串与自然数的有效一一对应。给定 x，生成机器 N_x：输入对应自然数 n 的串后，依次尝试 $t = 0, 1, 2, \ldots$，找到 $R(x,n,t)$ 成立就接受。

$$L(N_x) = \Sigma^* \iff \forall n\, \exists t\, R(x,n,t) \iff x \in L.$$

构造代码时只需把参数 x 写进程序，故 $x \mapsto \langle N_x \rangle$ 是全可计算归约。结合 Π_2 上界，得到完全性。

例 5.23　有限性与无限性的量词形式

定义 $\mathrm{FIN}_{\mathrm{TM}} = \{\langle M \rangle : L(M) \text{ 有限}\}$。有限字母表上，语言有限等价于成员长度有上界，因此

$$\langle M \rangle \in \mathrm{FIN}_{\mathrm{TM}} \iff \exists b\, \forall x\, \forall t\, \big(\mathrm{Acc}(M,x,t) \Rightarrow |x| \le b\big).$$

所以有限性在 Σ_2；其补 $\mathrm{INF}_{\mathrm{TM}}$ 在 Π_2，也可直接写成 $\forall b\, \exists x, t\, [|x| > b \wedge \mathrm{Acc}(M,x,t)]$。

有限性的完全性构造

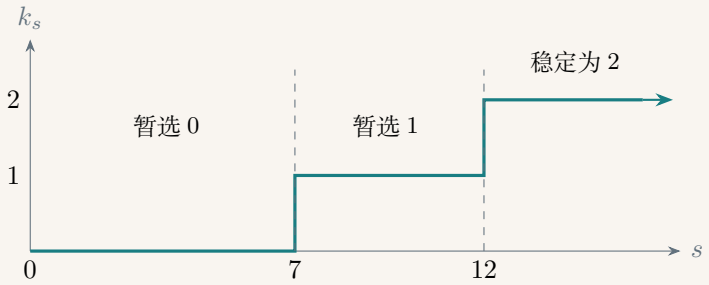
设 $L \in \Sigma_2$，$x \in L \iff \exists n\, \forall m\, R(x,n,m)$。在阶段 s，找最小的 $n \le s$，使所有 $m \le s$ 都满足 $R(x,n,m)$；若没有这样的 n，令 $k_s = s+1$，否则令 k_s 为这个最小候选。然后枚举标准序中的前 k_s 个串。

若存在最小真正见证 n_0，比它小的候选各自有一个失败证据，足够大的阶段会排除它们；此后 $k_s = n_0$。前期有限多个阶段也只输出有限多个串，所以枚举语言有限。若不存在真正见证，则对每个固定界限 b，前 b 个候选最终全被排除，故 k_s 趋于无穷，输出语言为 Σ^*。

这一枚举器可以有效转换为识别器，因此给出 $L \le_m \mathrm{FIN}_{\mathrm{TM}}$。有限性为 Σ_2 完全语言，无限性为 Π_2 完全语言。

例 5.24　候选见证如何被淘汰

假设候选 $n=0$ 首次在 $m=7$ 失败，$n=1$ 首次在 $m=12$ 失败，而 $n=2$ 对所有 m 都成功。在阶段 $s<7$ 时暂选 0；随后暂选 1；到 $s \ge 12$ 后，最小候选稳定为 2。虽然无法在某个有限时刻知道“以后永远不会再失败”，稳定本身足以让枚举结果保持有限。



5.9 典型判定问题对照

下表统一使用固定输入字母表，A 表示成员资格，E 表示空性，ALL 表示满性。完全性均指可计算的映射归约。

问题	结论	主要方法
$A_{\mathrm{DFA}}, E_{\mathrm{DFA}}$	可判定	模拟、有限图可达性
$\mathrm{EQ}_{\mathrm{DFA}}$	可判定	对称差 DFA 的空性
$A_{\mathrm{CFG}}, E_{\mathrm{CFG}}$	可判定	CYK、变量标记
A_{LBA}	可判定	有限格局图搜索
$A_{\mathrm{TM}}, \mathrm{HALT}_{\mathrm{TM}}$	Σ_1 完全	通用模拟、双向归约
$\overline{E_{\mathrm{TM}}}$	Σ_1 完全	寻找被接受的串
E_{TM}	Π_1 完全	对非空性取补
$\mathrm{ALL}_{\mathrm{CFG}}$	Π_1 完全	错误计算历史
$\mathrm{EQ}_{\mathrm{CFG}}, \mathrm{SUB}_{\mathrm{CFG}}$	Π_1 完全	反例枚举、满性归约
$E_{\mathrm{LBA}}, \mathrm{ALL}_{\mathrm{LBA}}$	Π_1 完全	合法或错误历史检查
$\mathrm{DISJ}_{\mathrm{TM}}$	Π_1 完全	同时接受的见证、空性归约
$\mathrm{ALL}_{\mathrm{TM}}, \mathrm{TOT}_{\mathrm{TM}}$	Π_2 完全	$\forall x \exists t$、通用搜索
$\mathrm{EQ}_{\mathrm{TM}}, \mathrm{SUB}_{\mathrm{TM}}$	Π_2 完全	满性归约、逐步接受模拟
$\mathrm{FIN}_{\mathrm{TM}}$	Σ_2 完全	长度上界、候选稳定构造
$\mathrm{INF}_{\mathrm{TM}}$	Π_2 完全	有限性的补
$L(M)$ 是否正则 / CFL	不可判定	Rice 定理

其中包含问题的 Π_2 上界可写为

$$\forall x\, \forall t\, \exists s\, \big(\mathrm{Acc}(M_1, x, t) \Rightarrow \mathrm{Acc}(M_2, x, s)\big).$$

等价问题要求两个方向都成立，利用 Π_2 对交封闭即可。表中的层次来自具体的上界证明和完全性归约，不能仅由“问题看起来更难”来确定。

第 6 章　可计算性高级专题

机器编码既可以作为数据被处理，也可以被另一台机器执行。本章利用这一点构造自我引用，再通过谕示研究相对可计算性，最后用最短程序长度讨论有限串的随机性。

6.1　自我复制与递归定理

用 $\varphi_e(x)$ 表示编码为 e 的程序在输入 x 上计算的部分函数。符号 $\simeq$ 表示两边都未定义，或都定义且输出相同。对只研究语言的机器，也可以比较接受的语言是否相同；本节定理采用部分函数这一更强的形式。

四种表述

1. **自我复制**。可以有效构造机器 SELF，使它在任意输入上输出自身的完整编码并停机；构造中可以保留原输入供后续计算使用。

2. **递归定理**。对任意二元部分可计算函数 $t(x,y)$，可以有效构造机器 M，使

$$\varphi_{\langle M\rangle}(x) \simeq t(x,\langle M\rangle).$$

   若 t 为全函数，则这个 M 也处处停机。

3. **自我引用**。在描述机器算法时，可以使用“取得本机编码 $\langle M\rangle$，然后继续计算”的步骤；该描述能够转成普通 TM。

4. **不动点定理**。对任意把程序编码映到程序编码的全可计算函数 f，存在编码 e，使

$$\varphi_e \simeq \varphi_{f(e)}.$$

   这里相同的是计算行为，两个代码串本身可以不同。

这些表述的联系依赖程序能够被有效编码、组合和部分固定参数。下面给出具体构造，尤其说明如何把自我复制的方法扩展到任意后续计算。

直接构造自我复制

先定义一个全可计算的代码生成函数 q。给定有限串 z，$q(z)$ 输出程序 P_z 的代码；P_z 的功能是把常量 z 写到工作区并停机。生成代码只需要把 z 放入固定的打印模板。

再用 $c(a,b)$ 表示“先运行代码 a，再将工作区交给代码 b”这一组合程序的代码。c 也全可计算：有限代码块可以通过状态重命名和控制转移连接。原输入可以另存在一个轨道上。

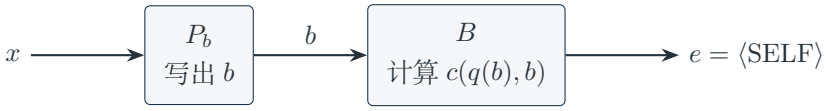
构造程序 B，其工作区输入为串 z，执行：

$$\text{计算} a = q(z), \qquad \text{输出} c(a,z).$$

先完成 B 的代码，记为 $b=\langle B\rangle$；然后计算 $a=q(b)$，最后令 $e=c(a,b)$。编码为 e 的机器先由 P_b 输出 b，再把 b 交给 B。后者输出

$$c(q(b),b)=c(a,b)=e.$$

这就是它自己的编码。整个定义顺序是先定义通用的 q,c，再定义 B，最后组成 SELF，每一步都是有限的代码生成。



例 6.1　一段直接打印自身的程序

下面的 Python 程序输出与自身相同的一行源代码，末尾带换行：

```
s='s=%r;print(s%%s)';print(s%s)
```

字符串 s 保存代码模板。格式符 $\%r$ 将模板作为带引号的字符串放回程序中，$\%\%$ 产生一个字面的百分号。模板既提供“数据是什么”，也提供“怎样把数据写进程序”的规则，这与上面的打印代码生成函数和组合函数承担的工作相对应。

由自我复制构造递归定理

给定计算 $t(x,y)$ 的程序 T。在刚才的构造中，定义新的后半段 B_T：输入工作串 z，先算 $y=c(q(z),z)$，然后在保留的原输入 x 上运行 $T(x,y)$。

令 $b_T=\langle B_T\rangle$，$a_T=q(b_T)$，并令 $e=c(a_T,b_T)$。这台机器先写出 b_T，再由 B_T 算出

$$y=c(q(b_T),b_T)=e,$$

随后运行 $T(x,e)$，所以 $\varphi_e(x)\simeq t(x,e)$。

这个过程把后续程序 T 一并纳入整体编码的重建，因此得到的是最终机器自己的代码。它使用了自我复制的同一套有效构造，扩展了复制后可以执行的任务。

递归定理、自我引用与自我复制

假设希望写一台机器，其算法在输入 x 时会用到自己的编码。先把“自己的编码”临时替换为一个额外参数 y，就得到普通二元程序 $T(x,y)$。递归定理提供 M，使它的行为恰好是 $T(x,\langle M\rangle)$。这证明自我引用语法可以实现。

反过来，使用自我引用写“取得自己的编码并打印”，立即得到自我复制机器。因此已经形成

$$\text{自我复制构造}\Longrightarrow\text{递归定理}\Longrightarrow\text{自我引用}\Longrightarrow\text{自我复制}.$$

递归定理推出不动点定理

给定全可计算的程序变换 f，定义二元部分可计算过程

$$t(x,y)=\text{先计算}f(y)\text{，再模拟该程序在}x\text{上运行}.$$

由递归定理，存在 e 使

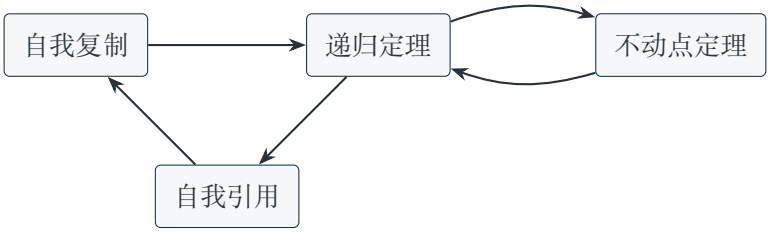
$$\varphi_e(x)\simeq t(x,e)\simeq\varphi_{f(e)}(x).$$

由于这对每个 x 都成立，e 就是行为意义下的不动点。

不动点定理推出递归定理

给定 $t(x,y)$，定义代码生成函数 $f(y)$：输出“输入 x 后运行 $t(x,y)$”的程序代码。这里 y 被写成代码中的常量。生成代码的函数 f 总停机，即使生成出来的程序在某些输入上可能不停机。

不动点定理给出 e，满足 $\varphi_e \simeq \varphi_{f(e)}$；而后者在 x 上正是 $t(x,e)$。于是 $\varphi_e(x) \simeq t(x,e)$，得到递归定理。



例 6.2 计算输入长度与自身代码长度之和

取 $t(x,y)$ 为整数 $|x|+|y|$ 的二进制表示。递归定理给出一台 M，使它在输入 x 上输出 $|x|+|\langle M\rangle|$。构造并不需要事先猜出最终代码长度；机器在运行时通过自我引用得到代码，再计算长度。

例 6.3 代码变换的不动点

令 f 给每个程序加上一段无效的预处理，例如先在工作带写一个标记，再擦掉，随后运行原程序。代码长度和状态数可能增加，但某些甚至全部程序的计算函数仍不变。
不动点定理对任意全可计算程序变换都保证至少一个行为不动点。因此不存在一个全可计算函数，能够把每个程序都改成与原程序计算不同函数的程序。

6.2 递归定理的不可判定性应用

例 6.4 重新证明接受问题不可判定

假设 H 判定 A_{TM}。通过递归定理构造 M：输入 x 后取得自己的编码 e，运行 $H(\langle M,x\rangle)$，并翻转接受与拒绝结果。于是每个 x 都满足

$$M \text{ 接受} x \iff H(\langle M,x\rangle) \text{ 拒绝} \iff M \text{ 不接受} x,$$

矛盾。自我引用让这台机器直接对自身行为提出问题。

例 6.5 最短机器编码集合不可识别

固定编码方式，定义

$$\mathrm{MIN}_{\mathrm{TM}} = \{\langle M\rangle : \text{不存在更短编码的机器识别} L(M)\}.$$

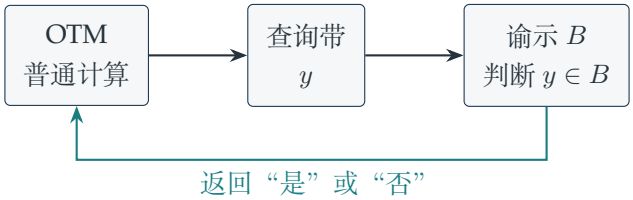
允许相同长度有多个最短描述。因为可识别语言有无限多个，而任意长度界内只有有限多个代码，所以这些最短编码长度无界。
假设 $\mathrm{MIN}_{\mathrm{TM}}$ 可识别，就存在枚举器 E。利用自我引用构造机器 C：取得自己的代码，运行 E，等待它输出一个代码长度大于 $|\langle C\rangle|$ 的机器 D，随后在原输入 x 上模拟 $D(x)$。由于最短编码长度无界，等待一定结束，且选择的同一个 D 与输入 x 无关。因此 $L(C)=L(D)$，但 $|\langle C\rangle| < |\langle D\rangle|$，与 D 的最短性矛盾。故该集合不可识别。

例 6.6　无法有效列出无限多个最短代表

上一个论证还证明：$\mathrm{MIN}_{\mathrm{TM}}$ 不含无限可识别子集。若只枚举其中某个无限子集，代码长度仍无界，机器 C 的等待和矛盾仍然成立。这个证明方式会在随机串的讨论中再次出现。

6.3　谕示图灵机与图灵归约

谕示图灵机（OTM）拥有普通工作带、查询带以及特殊查询状态。相对于集合 B 的机器记作 M^B：当它把串 y 写到查询带并进入查询状态时，谕示立即返回 $y \in B$ 的真假，机器随后沿“是”或“否”的分支继续运行。



这是相对计算的数学定义：把集合 B 的成员资格作为可用信息。若 B 本身不可判定，就不能用普通 TM 把这一查询步骤全部实现。

若存在一台相对于 B、在每个输入上都停机的 OTM 判定 A，称 A 相对于 B 可判定，或 A 图灵归约到 B，记 $A \leq_T B$。每个输入的运行中只能提出有限次查询，但查询次数可以随输入变化，后续查询也可以依赖先前回答。

基本性质

映射归约蕴含图灵归约：先计算 $f(x)$，查询一次 B，直接返回答案即可。图灵归约具有自反性和传递性；传递性通过用 C-谕示程序替换每次对 B 的查询实现。由于外层每次运行只有有限次查询，每次内层计算又会停机，组合也会停机。

若 B 可判定，则每次谕示查询都可用其判定器实现，所以 $A \leq_T B$ 推出 A 可判定。对固定 B，相对于 B 可判定的语言对并、交、补、连接、星与反转都封闭，构造与普通判定器相同。

例 6.7　每个语言与其补图灵等价

有 B 的谕示后，判定 $\overline{B}$ 只需查询 $x \in B$，再翻转答案。因此

$$B \equiv_T \overline{B}.$$

特别地，$A_{\mathrm{TM}} \equiv_T \overline{A_{\mathrm{TM}}}$。但 $A_{\mathrm{TM}} \not\leq_m \overline{A_{\mathrm{TM}}}$：若有这种映射归约，由目标补可识别及向下封闭，$A_{\mathrm{TM}}$ 也会补可识别，从而可判定，矛盾。图灵归约允许对谕示回答作后续处理，能力因此更强。

例 6.8　用 A_{TM} 谕示判定空性

给定 M，有效构造机器 N_M：交错模拟 M 在所有串上的运行，看到任何接受就接受。然后只查询一次

$$\langle N_M, \varepsilon \rangle \in A_{\mathrm{TM}} \ ?$$

例 6.8　用 A_{TM} 谕示判定空性（续）

若回答“是”，说明 $L(M)$ 非空；否则为空。所以 $E_{\mathrm{TM}} \le_T A_{\mathrm{TM}}$。这里询问的是一个专门构造的“搜索是否成功”问题，一次查询即可概括无限多条候选运行。

相对停机问题与跳跃

对任意谕示 B，定义

$$B' = \{\langle M, x\rangle : M^B \text{ 在} x \text{ 上停机}\}.$$

称 B' 为 B 的图灵跳跃。它相对于 B 可识别，但相对于 B 不可判定：若有 B-谕示判定器 H^B，构造 D^B，在输入自身编码时根据 H^B 的回答执行相反的停机行为，就得到与普通停机问题相同的对角矛盾。

另外 $B \le_T B'$：构造程序查询 $y \in B$，回答“是”就停机，否则循环，再问它是否停机。因此 B' 比 B 严格更强。定义

$$0^{(0)} = \varnothing, \qquad 0^{(n+1)} = (0^{(n)})'.$$

$0'$ 与 A_{TM} 图灵等价，也彼此可映射归约；每次跳跃都得到更强的谕示。

6.4　谕示与 Kleene 谱系

用 Σ_1^B 表示相对于 B 可识别的语言类，用 Π_1^B 表示其补类，用 Δ_1^B 表示相对于 B 可判定的语言类。交替模拟的证明仍然适用，所以

$$\Delta_1^B = \Sigma_1^B \cap \Pi_1^B.$$

Post 定理的谱系刻画

对每个 $n \ge 0$，

$$\Sigma_{n+1} = \Sigma_1^{0^{(n)}}, \qquad \Pi_{n+1} = \Pi_1^{0^{(n)}}, \qquad \Delta_{n+1} = \Delta_1^{0^{(n)}}.$$

特别地，$\Sigma_2 = \Sigma_1^{A_{\mathrm{TM}}}$、$\Pi_2 = \Pi_1^{A_{\mathrm{TM}}}$、$\Delta_2 = \Delta_1^{A_{\mathrm{TM}}}$。

这里上标是一个具体谕示集合。使用某层的任意完全语言可以替代相应跳跃，因为这些完全语言彼此可映射归约；单独写一个语言类作为谕示，需要额外解释选取了哪个集合。

证明 $\Sigma_2 \subseteq \Sigma_1^{A_{\mathrm{TM}}}$

设

$$x \in L \iff \exists y \forall z\, R(x, y, z),$$

R 可判定。对固定 x, y，构造普通机器 $N_{x,y}$，搜索一个使 $R(x, y, z)$ 为假的 z，找到就接受。询问 $\langle N_{x,y}, \varepsilon\rangle \in A_{\mathrm{TM}}$ 便能知道是否存在反例。因此 OTM 可以逐个枚举 y，每次问一次谕示；第一次得到“不存在反例”时接受。

若确实存在合适的 y，它最终会被找到；若不存在，则一直搜索。这给出相对于 A_{TM} 的识别器。

例 6.9　相对于 A_{TM} 识别有限性

有限性具有形式 $\exists b\,\forall x,t\,[\mathrm{Acc}(M,x,t)\Rightarrow |x|\le b]$。依次试 $b=0,1,2,\ldots$；对每个 b，构造一台寻找长度大于 b 的接受串的普通机器，再询问它是否会接受。一旦谕示回答“不会”，这个 b 就是长度上界，OTM 接受。

无限语言不会出现成功的长度界，所以这个过程只是相对识别器。要相对判定，还需要额外证明其补也能相对识别。

证明 $\Sigma_1^{A_{\mathrm{TM}}}\subseteq\Sigma_2$

设 OTM $M^{A_{\mathrm{TM}}}$ 接受 x。一次接受运行只进行有限步、提出有限次查询，因此可以用一个有限串 h 描述整个运行以及所有猜测的谕示回答。

对被回答“是”的查询 q，还提供一个接受时间或接受历史作为见证；对被回答“否”的查询 r，要求任意有限时间内都没有接受。定义 $\mathrm{Acc}(q,t)$ 为编码 q 所描述的普通机器输入对在前 t 步内接受；格式不合法时取假。于是存在接受运行可以写成

$$\exists h,\vec{u}\,\forall t\left[\begin{array}{l}\mathrm{Path}(M,x,h)\\ \wedge\displaystyle\bigwedge_{q\in Y(h)}\mathrm{Acc}(q,u_q)\\ \wedge\displaystyle\bigwedge_{r\in N(h)}\neg\,\mathrm{Acc}(r,t)\end{array}\right].$$

Path 检查记录在所写答案下是否为合法接受路径；$Y(h),N(h)$ 是记录中的正、负查询集合。它们都有限，括号内可判定。将 $h,\vec{u}$ 编成一个存在变量，便得到 $\exists\forall$ 形式。

若真有接受运行，可选取它及所有正回答的真实见证；反过来，公式成立保证每个正、负回答都真实，因而记录确实对应一条接受运行。两个方向一起证明 $\Sigma_2=\Sigma_1^{A_{\mathrm{TM}}}$。对更高层重复相对化的同一论证即可得到一般形式；取补和交得到 Π,Δ 的关系。

谱系的严格性

$0^{(n)}$ 可由它自己的谕示判定，但相对跳跃 $0^{(n+1)}$ 不能由 $0^{(n)}$ 判定。结合 Post 定理，这给出

$$0^{(n+1)}\in\Sigma_{n+1}\setminus\Delta_{n+1}.$$

它相对于 $0^{(n)}$ 可识别而不可判定，因此不能同时相对补可识别，故也不属于 Π_{n+1}。取补得到另一侧的分离。

每个 n 层集合都可以借助 $0^{(n)}$ 判定，因而进入 Δ_{n+1}。此外，Δ_{n+1} 中也有既不在 Σ_n、也不在 Π_n 的集合：把 $0^{(n)}$ 与其补用不同前缀拼在一起，

$$D=\{0x:x\in 0^{(n)}\}\cup\{1x:x\notin 0^{(n)}\}.$$

D 可由 $0^{(n)}$ 判定。若 $D\in\Sigma_n$，取前缀 1 的可计算原像会使 $\overline{0^{(n)}}\in\Sigma_n$；若 $D\in\Pi_n$，取前缀 0 又会使 $0^{(n)}\in\Pi_n$，都矛盾。这里使用相对停机集合的标准完全性：$0^{(n)}$ 是 Σ_n 完全的，来自 Post 定理中通用相对模拟的有效构造。

因此图中的包含确实严格。但对具体语言进行归约时，仍需给出转换函数或使用已证明的完全性；类之间的包含不替代语言之间的归约证明。

6.5 Kolmogorov 复杂性

一个串可以由不同程序输出。固定一台最优通用描述机 U，定义

$$K_U(x) = \min\{|p| : U(p) \text{ 停机并输出} x\}.$$

固定 U 后简写为 $K(x)$。这里使用普通描述复杂性，程序不要求构成前缀自由集合；有些文献把它记作 $C(x)$，并将 $K(x)$ 留给前缀复杂性。以下统一采用当前定义。

“最优”指 U 能模拟任何固定描述机 D，只付出一个与输出串无关的常数长度开销。给 D 一个固定解释器前缀，再附上输入程序即可实现。因此对每台 D 存在常数 c_D，满足

$$K_U(x) \le K_D(x) + c_D.$$

若 U, V 都是固定的最优通用机，两方向模拟给出

$$|K_U(x) - K_V(x)| \le c_{U,V}.$$

这称为不变性定理，也称相对性原理。它比较的是改变描述系统带来的固定开销。

例 6.10　重复串具有短描述

串 0^n 可以由“读入二进制整数 n，打印 n 个 0”的固定程序输出。程序只需要编码 n，所以

$$K(0^n) \le \lfloor \log_2 n \rfloor + c \quad (n \ge 1),$$

常数 c 吸收解释器与固定代码。串长为 n，描述长却只随 $\log n$ 增长。类似地，$(01)^n$ 也有这种短描述。

例 6.11　外观不规则并不决定复杂性

取某个固定可计算序列的前 n 位。即使这段字符串在视觉上很杂乱，它仍能由“运行该序列的生成算法，输出前 n 位”描述，复杂性至多为 $\log n + O(1)$。因此这里的随机性以是否存在短程序为准，具体描述必须包含生成算法和参数。

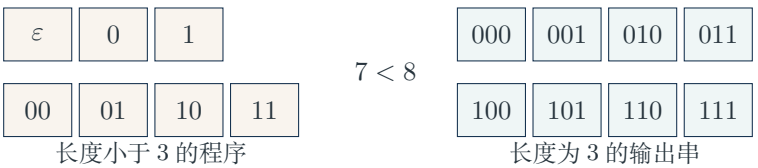
不可压缩串的计数

长度恰为 n 的二进制串有 2^n 个，长度小于 n 的二进制程序总数为

$$1 + 2 + \cdots + 2^{n-1} = 2^n - 1.$$

每个停机程序只有一个输出，所以这些短程序不能覆盖全部 n 位串。至少有一个 x 满足 $|x| = n$、$K(x) \ge n$。

更一般地，对整数 $c \ge 0$、$n \ge c$，满足 $K(x) < n - c$ 的 n 位串至多有 $2^{n-c} - 1$ 个，比例严格小于 2^{-c}。所以至少 $1 - 2^{-c}$ 的 n 位串具有复杂性不小于 $n - c$。



长度小于 3 的程序共 7 个　　长度为 3 的输出串共 8 个

每个停机程序至多给出一个输出。即使七个短程序输出都不同，也无法覆盖右侧八个串。

例 6.12　压缩十位以上的串占多少

在所有 n 位串中，能够用长度小于 $n-10$ 的程序描述的比例小于 $2^{-10}=1/1024$。这个结论不依赖某种特定压缩算法，而是直接计算所有足够短的描述可能覆盖多少个输出。

6.6　$K(x)$ 的不可计算性

最短描述长度不可计算

不存在一个对每个输入串 x 都停机并输出准确数值 $K(x)$ 的算法。

证明。假设可以计算 K。给定整数 n，按标准序搜索第一个满足 $K(x)\geq n$ 的串，记为 x_n。计数论证保证这样的串存在；每次复杂性计算都停机，所以搜索最终结束。

于是 x_n 可以由一个固定程序和整数 n 描述：运行上述搜索并打印结果。因而

$$K(x_n)\leq \lfloor \log_2 n\rfloor + c.$$

但按选取方式，$K(x_n)\geq n$。当 n 足够大时，$\log_2 n + c < n$，得到矛盾。 □

从上方逼近

虽然 K 不可计算，但可以不断改善其已知上界。先采用“原样打印 x”的程序作为一个上界，然后交错模拟其他程序；每发现一个更短程序输出 x，就降低当前记录。最短程序最终会运行到停机，因此上界最终稳定在 $K(x)$。一般无法判断何时已经稳定，因为某个还没有停机的更短程序可能将来才输出 x。

例 6.13　压缩文件长度给出的是一种上界

若某个固定解压程序能从长度为 m 的数据恢复 x，则 $K(x)\leq m+c$，其中 c 为描述解压程序的固定开销。找到更好的压缩方法能降低这个上界，但它并不自动证明不存在其他更短的描述。

6.7　随机串语言 RANDOM

固定上面的 U，定义

$$\mathrm{RANDOM}=\{x\in\{0,1\}^{*} : K(x)\geq |x|\}.$$

这里“随机”表示在当前描述系统下不可压缩。计数论证说明每个长度都有这样的串，所以 RANDOM 无限。具体某个短串是否属于它会受到通用机选择的常数开销影响；长串的复杂性估计则可用不变性定理比较。

RANDOM 不含无限可识别子集

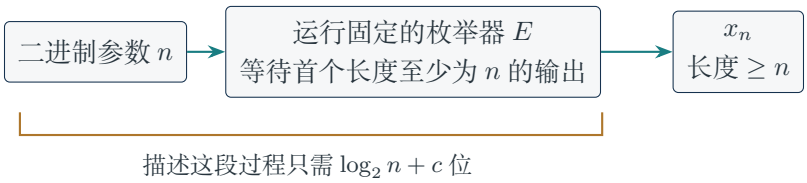
不存在一个枚举器，能够只输出 RANDOM 中的串，并输出无限多个互不相同的串。因此 RANDOM 不可识别。

证明。假设有这样的枚举器 E。给定 n，运行 E，等待第一个长度至少为 n 的输出，记为 x_n。无限多个不同串的长度必然无界，所以等待会结束。

“运行固定的 E，等待首个长度至少为 n 的输出”是一段固定算法，加上二进制参数 n 后，描述长度至多为 $\log_2 n + c$。因此

$$K(x_n) \le \log_2 n + c < n \le |x_n|$$

对充分大的 n 成立，与 $x_n \in \mathrm{RANDOM}$ 矛盾。由于 RANDOM 本身无限，若它可识别就可枚举，也会产生这种矛盾。 □



当 n 足够大，左侧的短描述就能输出右侧长串，与“每个输出都不可压缩”的假设冲突。

例 6.14　为什么选择“长度至少为 n”

假设枚举器只输出长度为 $2, 4, 8, 16, \ldots$ 的串。等待一个长度恰为 7 的串可能永远没有结果；等待第一个长度至少为 7 的串则一定成功。证明需要的只是输出长度不小于 n，这样短描述与串长之间仍然产生矛盾。

补语言可识别

一个串可压缩，意味着存在长度小于 $|x|$ 的程序，最终停机并输出它：

$$x \notin \mathrm{RANDOM} \iff \exists p, t\ \big[|p| < |x| \ \wedge\ U(p) \text{ 在前} t \text{ 步内停机并输出} x\big].$$

右侧是存在量词加可判定关系。因此可压缩串组成可识别语言，RANDOM 属于 $\Pi_1 = \mathrm{coRE}$。结合上面的证明，

$$\mathrm{RANDOM} \in \mathrm{coRE} \setminus \mathrm{RE}.$$

例 6.15　找到压缩证据与确认不可压缩

给定 x，同时运行全部长度小于 $|x|$ 的程序。如果其中一个输出 x，就得到了具体压缩证据，可以停机确认 x 不随机。若始终没有找到，其中某些程序可能仍在运行；有限次检查无法由这个过程确认它们将来永远不会输出 x。这与可识别和补可识别的区别一致。